

Game Theory Algorithms

A comprehensive guide for SDE placements, competitive programming, and online assessments.

BASIC WINNING/LOSING STATES

1. Overview

In combinatorial game theory, every position in a finite, deterministic, two-player perfect-information game can be classified as either a **winning state** or a **losing state**. A winning state is one where the player whose turn it is can force a win with optimal play. A losing state is one where no matter what the player does, the opponent can force a win.

This classification forms the foundation of all impartial combinatorial game analysis.

2. Intuition

Simple explanation: A position is winning if you can move to a losing position for the opponent. A position is losing if every move leads to a winning position for the opponent.

Analogy: Imagine standing on a staircase. If you can step onto a step labelled "LOSE" for the other person, your current step is "WIN". If every step you can go to is labelled "WIN" for the other person, your current step is "LOSE". You label backwards from terminal positions.

Step-by-step reasoning:

1. Terminal positions (where no moves are possible) are losing states — the player to move has lost.
2. For any non-terminal position:
 - If there exists at least one move to a losing state → current state is winning.
 - If all moves lead to winning states → current state is losing.
3. Propagate backwards from terminal positions using recursion/DP.

Why it works: This is a backward induction principle. Under optimal play, players always choose winning moves. If you can hand the opponent a losing position, you win. If all your options give the opponent winning positions, you lose.

3. When to Use It

- Two-player turn-based games with perfect information
- No randomness involved
- Finite game tree (game must end)
- Impartial games (both players have the same moves available from any given state)
- Problems asking "can the first player win?"
- Problems with "optimal play" phrasing

Common trigger phrases: "optimal play", "first player wins", "winning strategy", "Alice and Bob take turns", "determine the winner", "terminal position", "no moves left loses".

4. When Not to Use It

- Games with randomness or hidden information (poker, dice games)
- Non-zero-sum games (both can "win")
- Infinite games (no termination guarantee)
- Games with draws that repeat states (chess, tic-tac-toe — need additional handling)
- Partizan games where players have different move sets (use other tools)
- When the state space is too large to enumerate — need Grundy numbers / Sprague-Grundy instead

5. Core Concepts

Terminal State

A state with zero moves available. By convention, it is a losing state for the player whose turn it is.

Winning State (N-position)

A state from which the player to move can force a win. There is at least one move to a losing state.

Losing State (P-position)

A state from which the player to move will lose if the opponent plays optimally. All moves lead to winning states.

Optimal Play

Both players always choose the best move available. This means:

- From a winning state, the player picks a move that goes to a losing state.
- From a losing state, all moves go to winning states — the player is doomed.

DP Transition

```
win[u] = any(move -> win[v] == false for all v in next(u))
```

Game Tree

A tree where nodes are game states and edges are moves. Terminal nodes are leaves. The classification labels each node as WIN or LOSE.

6. Step-by-Step Algorithm

1. **Identify terminal states** — states where no further moves can be made. Mark them as **LOSE**.
2. **Process states in reverse topological order** (from terminal towards start):
 - If the game naturally allows DP (like take-away games), process states 0..N.
3. **For each state:**

- Look at all possible next states.
- If any next state is LOSE → mark current as WIN.
- If all next states are WIN → mark current as LOSE.

4. **Answer:** Check if the starting state is WIN (first player wins) or LOSE (first player loses).

7. Dry Run

Problem: Take-away game. There are n stones. Players can take 1, 2, or 3 stones per turn. Player who takes the last stone wins.

Let $n = 5$.

Stones Left	Possible Moves	Next States	Classification
0	—	—	LOSE (no move)
1	take 1	0 (LOSE)	WIN
2	take 1 or 2	1 (WIN), 0 (LOSE)	WIN
3	take 1, 2, 3	2 (WIN), 1 (WIN), 0 (LOSE)	WIN
4	take 1, 2, 3	3 (WIN), 2 (WIN), 1 (WIN)	LOSE
5	take 1, 2, 3	4 (LOSE), 3 (WIN), 2 (WIN)	WIN

Result: First player wins when $n = 5$.

Observation: States where $n \% 4 == 0$ are losing. All others are winning.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Determine if the first player wins a take-away game
// where a player can take 1..k stones from a pile of n stones.
// Last move wins.
bool isWinningState(int n, int k) {
    // dp[i] = true if i stones is a winning state
    vector<bool> dp(n + 1, false);

    // Base case: 0 stones = losing state
    dp[0] = false;

    for (int i = 1; i <= n; i++) {
        for (int take = 1; take <= k && take <= i; take++) {
            if (!dp[i - take]) { // can move to losing state
                dp[i] = true;
                break;
            }
        }
    }
}
```

```

        return dp[n];
    }

    // For the specific case k=3 (take 1..3 stones)
    bool isWinningStateSimplified(int n) {
        return n % 4 != 0;
    }

    int main() {
        int n = 5, k = 3;
        cout << (isWinningState(n, k) ? "First player wins\n" : "First player
loses\n");
        return 0;
    }

```

9. Python Implementation

```

def is_winning_state(n: int, k: int) -> bool:
    """Determine if first player wins a take-away game (take 1..k stones)."""
    dp = [False] * (n + 1)
    # dp[0] = False (terminal, losing)

    for i in range(1, n + 1):
        for take in range(1, k + 1):
            if take > i:
                break
            if not dp[i - take]:
                dp[i] = True
                break

    return dp[n]

# Simplified for k = 3
def is_winning_simple(n: int) -> bool:
    return n % 4 != 0

if __name__ == "__main__":
    n, k = 5, 3
    print("First player wins" if is_winning_state(n, k) else "First player loses")

```

10. Code Explanation

Key parts of the code:

1. **dp array:** `dp[i]` stores whether `i` stones is a winning state.
2. **Base case:** `dp[0] = false` — no stones means the player to move has already lost.
3. **Transition loop:** For each `i`, try all legal moves (taking 1..k stones).

- If any move leads to a losing state `dp[i - take] == false`, then `dp[i] = true`.
- Break early once a winning move is found (optimization).

4. **Final answer:** `dp[n]` tells if the starting player wins.

Edge cases handled:

- `n = 0`: returns `false` (correct — first player loses).
- `k > n`: inner loop caps at `i` via `take <= i`.
- Large `n`: `O(n*k)` may be slow; use the mathematical pattern if possible.

11. Complexity Analysis

Case	Time	Space
DP (general k)	$O(n \times k)$	$O(n)$
DP (optimized with pattern)	$O(n)$	$O(1)$
Pattern-based (if periodic)	$O(1)$	$O(1)$

Best case: Pattern emerges (e.g., `n % (k+1)`) — $O(1)$. **Worst case:** State space too large — need Grundy numbers or game decomposition.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Take-away game	"take 1..k stones", last move wins	DP up to <code>n</code> , check <code>n % (k+1)</code>	Stone Game, Nim (simple)
Multi-pile	"multiple piles", "each turn choose one pile"	XOR (Nim) or Grundy per pile	Nim Game
Reachability game	Move on a DAG, reach terminal	DP on graph (reverse topological)	Chessboard moves
Subtraction set	Can remove from set <code>S</code> (not contiguous)	DP with set of moves	Subtraction Game

13. Common Mistakes

- Marking terminal states as winning — they are losing (player to move cannot move and loses).
- Forgetting that both players play optimally — assuming the opponent will make mistakes.
- Only considering immediate win, not forcing the opponent into a losing position later.
- Not handling the case where no moves exist from a non-terminal state (should not happen in well-defined games).
- Using BFS instead of reverse topological order on game DAG.

14. Edge Cases

- `n = 0`: first player loses immediately.
- `n = 1`: if you can take 1, first player wins.

- $k \geq n$: player can take all stones in one move — always winning (except $n=0$).
- Large n with periodic pattern: don't DP, use mathematical formula.
- Game where last move loses (misère): terminal now means last move was bad; adjust base case.

15. Variations

Misère Game

Last player to move loses. The terminal state (no moves) is now a **winning** state for the player who cannot move. The DP logic flips: `dp[0] = true`. Everything else is same transition.

Subtraction Set Game

Instead of contiguous range $1..k$, the allowed moves are from a set $S = \{a, b, c, \dots\}$. Same DP, just iterate over set elements.

Reachability / DAG Game

States form a DAG edges are moves. Compute win/lose via DP on reverse topological order using DFS + memoization.

16. Related Algorithms/Data Structures

Topic	Connection
Grundy Numbers	Generalizes win/lose to assign a numerical value (nimber) to each state.
Sprague-Grundy Theorem	Decomposes composite games into XOR of Grundy numbers.
Nim Game	Special case where Grundy number of a pile = pile size.
Minimax	Generalizes to non-zero-sum / partizan games with scores.
Game DP	Dynamic programming on state space.

17. Practice Problems

Easy

1. **Nim Game** — LeetCode 292. Take 1..3 stones, n stones. Pattern: $n \% 4 \neq 0$.
2. **Stone Game** — LeetCode 877. Even number of piles, pick from ends.

Medium

1. **Divisor Game** — LeetCode 1025. Alice and Bob pick divisors. DP win/lose on N .
2. **Predict the Winner** — LeetCode 486. Pick from ends, maximize score difference.
3. **Can I Win** — LeetCode 464. Numbers $1..maxChoosableInteger$, reach total.

Hard

1. **Stone Game III** — LeetCode 1406. Take 1..3 from a row, Alice and Bob.
2. **Cat and Mouse** — LeetCode 913. Graph game, mouse vs cat.

3. Avoid Stone Puzzle — Codeforces (search: game theory DP).

18. Interview Explanation

"Winning/losing states are the foundation of combinatorial game theory. We classify every position as winning or losing under optimal play. A position is winning if there is at least one move to a losing position. A position is losing if every move leads to a winning position. Terminal positions (no moves) are losing. We compute this bottom-up using DP, from terminal states backward to the starting state. The complexity is $O(\text{states} \times \text{moves per state})$. This is the base for Nim, Grundy numbers, and the Sprague-Grundy theorem."

19. Revision Notes

- **WIN:** exists move to LOSE
- **LOSE:** all moves go to WIN
- **Terminal = LOSE** (player to move cannot move)
- **DP:** Process from terminal $\rightarrow$ start (reverse topological order)
- **Periodic patterns:** Many take-away games have $n \% (k+1)$ pattern
- **Misère:** Flip terminal to WIN
- **Key question:** "Can first player force a win?"

20. Final Cheat Sheet

Item	Detail
When to use	Turn-based, perfect info, deterministic, impartial, finite
Main operation	Classify state as WIN/LOSE
Formula	WIN = any(move to LOSE); LOSE = all(move to WIN)
Code pattern	<code>dp[i] = any(!dp[i - move] for move in moves)</code>
Complexity	$O(\text{states} \times \text{moves per state})$
Edge cases	$n=0$, $k \geq n$, misère, periodic patterns
Trap	Don't mark terminal as WIN; don't ignore optimal play

NIM GAME

1. Overview

Nim is a classic impartial combinatorial game. There are several piles of stones. On each turn, a player chooses one pile and removes any positive number of stones from it (at least 1, up to the entire pile). The player who takes the last stone wins. Despite its simple rules, Nim is the central example in combinatorial game theory because the Sprague-Grundy theorem reduces any impartial combinatorial game to a Nim game.

2. Intuition

Simple explanation: Nim is just a XOR game. Compute the XOR of all pile sizes. If the XOR is non-zero, the first player wins. If zero, the first player loses.

Analogy: Think of each pile as a binary number. XOR (exclusive OR) is like adding without carry. If the XOR is zero, the position is "balanced" — the second player can mirror every move. If non-zero, the first player can make a move to balance it.

Step-by-step reasoning:

1. Consider two piles of equal size. First player takes X from one pile. Second player takes X from the other pile. This mirroring repeats until second player takes last. **Equal piles = losing.**
2. Now consider three piles: sizes are binary numbers. Compute XOR: $\text{pile1} \oplus \text{pile2} \oplus \text{pile3}$.
3. If $\text{XOR} = 0$, the position is a losing position (P-position) for the player to move.
4. If $\text{XOR} \neq 0$, the player can remove stones from one pile to make $\text{XOR} = 0$, handing the opponent a losing position.

Why it works: XOR is associative and has an inverse ("XOR is its own inverse"). From a non-zero XOR, there exists a pile where $(\text{pile XOR total_XOR}) < \text{pile}$, so you can reduce that pile to make total XOR zero. From a zero XOR, any move changes one pile and makes XOR non-zero, handing the opponent a winning position.

3. When to Use It

- Multiple piles of stones/tokens
- Each move: pick one pile, remove any positive number of stones
- Last move wins (normal play)
- Problem mentions "XOR strategy" or "Nim game"
- Any impartial game where each component is independent (apply XOR of Grundy numbers)

Common trigger phrases: "piles of stones", "remove any number of stones from one pile", "last move wins", "XOR of pile sizes", "Nim-sum".

4. When Not to Use It

- When there are constraints on what can be removed (a set of allowed counts, not any number) — use Grundy numbers per pile.
- Misère Nim (last move loses) — the strategy differs slightly: XOR rule holds except when all piles are size 1.
- Games where you can affect multiple piles in one move — breaks independence.
- Take-away games with a single pile — just check $n \% (k+1)$.
- Games with draws or cycles.

5. Core Concepts

Nim-Sum (XOR of all pile sizes)

The fundamental quantity. It determines the outcome:

- **Nim-sum = 0** → P-position (losing for player to move)
- **Nim-sum $\neq 0$** → N-position (winning for player to move)

Nim-Heap / Pile

An independent component of the game. Each pile contributes its size to the nim-sum.

Winning Move

From a non-zero nim-sum, find a pile where $\text{pile_size} \wedge \text{nim_sum} < \text{pile_size}$. Then reduce that pile to $\text{pile_size} \wedge \text{nim_sum}$. This makes the new nim-sum zero.

Terminal Position

All piles are zero. Nim-sum = 0. The player to move loses (cannot move).

Normal vs Misère Nim

- **Normal:** last move wins. XOR rule holds.
- **Misère:** last move loses. XOR rule holds EXCEPT when all piles are size 1. In that case, reverse the outcome.

6. Step-by-Step Algorithm

1. **Compute nim-sum:** $\text{nim_sum} = \text{xor of all pile sizes}$.
2. **If nim-sum == 0:** first player loses (under optimal play).
3. **If nim-sum != 0:** first player can win:
 - For each pile i with size $a[i]$:
 - If $(a[i] \wedge \text{nim_sum}) < a[i]$:
 - Winning move: reduce pile i to $a[i] \wedge \text{nim_sum}$.
 - Remove $a[i] - (a[i] \wedge \text{nim_sum})$ stones.
4. **Output:** Whether first player wins, and optionally the winning move.

7. Dry Run

Example: Piles = [3, 4, 5]

Step	Action	Piles	Nim-Sum
Start	—	[3, 4, 5]	$3 \wedge 4 \wedge 5 = 2$
Player 1	Find winning move: pile 5: $(5 \wedge 2)=7 > 5$ ❌; pile 4: $(4 \wedge 2)=6 > 4$ ❌; pile 3: $(3 \wedge 2)=1 < 3$ ✅. Reduce pile 3 to 1 (remove 2).	[1, 4, 5]	$1 \wedge 4 \wedge 5 = 0$
Player 2	Any move makes XOR non-zero. Suppose remove 3 from pile 4.	[1, 1, 5]	$1 \wedge 1 \wedge 5 = 5$
Player 1	$\text{nim_sum}=5$. Pile 5: $(5 \wedge 5)=0 < 5$ ✅. Reduce pile 5 to 0 (take all 5).	[1, 1, 0]	$1 \wedge 1 \wedge 0 = 0$
Player 2	Only move: take 1 from pile 1 (or pile 2).	[0, 1, 0]	$0 \wedge 1 \wedge 0 = 1$

Step	Action	Piles	Nim-Sum
Player 1	nim_sum=1. Pile 2: $(1^1)=0 < 1$ ✓. Take last stone.	[0, 0, 0]	0
Player 2	No moves. Player 2 loses, Player 1 wins.	—	—

Result: First player wins.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Check if the first player wins in normal Nim
bool firstPlayerWins(const vector<int>& piles) {
    int nim_sum = 0;
    for (int x : piles) nim_sum ^= x;
    return nim_sum != 0;
}

// Find any winning move and return {pile_index, stones_to_remove}
// If no winning move, returns {-1, -1}
pair<int, int> findWinningMove(vector<int>& piles) {
    int nim_sum = 0;
    for (int x : piles) nim_sum ^= x;
    if (nim_sum == 0) return {-1, -1}; // losing position

    for (int i = 0; i < (int)piles.size(); i++) {
        int new_size = piles[i] ^ nim_sum;
        if (new_size < piles[i]) {
            return {i, piles[i] - new_size};
        }
    }
    return {-1, -1}; // should not reach here
}

// Play optimal Nim as first player (prints moves)
void playNim(vector<int> piles) {
    cout << "Initial piles: ";
    for (int x : piles) cout << x << " ";
    cout << "\n";

    int turn = 0; // 0 = first player, 1 = second
    while (true) {
        int nim_sum = 0;
        for (int x : piles) nim_sum ^= x;

        if (nim_sum == 0) {
            cout << (turn == 0 ? "First" : "Second") << " player loses (no winning
```

```

move)\n";
    break;
}

bool move_found = false;
for (int i = 0; i < (int)piles.size(); i++) {
    int new_size = piles[i] ^ nim_sum;
    if (new_size < piles[i]) {
        int removed = piles[i] - new_size;
        cout << (turn == 0 ? "First" : "Second")
              << " player removes " << removed
              << " from pile " << i
              << " (" << piles[i] << " -> " << new_size << ")\n";
        piles[i] = new_size;
        move_found = true;
        break;
    }
}
if (!move_found) break;

// Check if all piles are zero
bool all_zero = all_of(piles.begin(), piles.end(), [](int x) { return x ==
0; });
if (all_zero) {
    cout << (turn == 0 ? "First" : "Second") << " player takes last stone
and wins!\n";
    break;
}

turn ^= 1;
}
}

int main() {
    vector<int> piles = {3, 4, 5};

    cout << (firstPlayerWins(piles) ? "First player wins\n" : "First player
loses\n");

    auto [idx, remove] = findWinningMove(piles);
    if (idx != -1) {
        cout << "Winning move: take " << remove << " from pile " << idx << "\n";
    }

    cout << "\n--- Full game simulation ---\n";
    playNim(piles);

    return 0;
}

```

9. Python Implementation

```

from typing import List, Tuple

def first_player_wins(piles: List[int]) -> bool:
    """Check if first player wins in normal Nim."""
    nim_sum = 0
    for x in piles:
        nim_sum ^= x
    return nim_sum != 0

def find_winning_move(piles: List[int]) -> Tuple[int, int]:
    """Return (pile_index, stones_to_remove) for a winning move, or (-1, -1)."""
    nim_sum = 0
    for x in piles:
        nim_sum ^= x
    if nim_sum == 0:
        return -1, -1

    for i, size in enumerate(piles):
        new_size = size ^ nim_sum
        if new_size < size:
            return i, size - new_size
    return -1, -1

def play_nim(piles: List[int]) -> None:
    """Simulate optimal Nim play."""
    piles = list(piles)
    print(f"Initial piles: {piles}")

    turn = 0 # 0 = first player
    while True:
        nim_sum = 0
        for x in piles:
            nim_sum ^= x

        if nim_sum == 0:
            print(f"{'First' if turn == 0 else 'Second'} player loses")
            break

        move_found = False
        for i, size in enumerate(piles):
            new_size = size ^ nim_sum
            if new_size < size:
                removed = size - new_size
                player = "First" if turn == 0 else "Second"
                print(f"{player} removes {removed} from pile {i} ({size} ->
{new_size})")
                piles[i] = new_size
                move_found = True
                break

```

```

        if not move_found:
            break

        if all(x == 0 for x in piles):
            player = "First" if turn == 0 else "Second"
            print(f"{player} takes last stone and wins!")
            break

        turn ^= 1

if __name__ == "__main__":
    piles = [3, 4, 5]
    print("First player wins" if first_player_wins(piles) else "First player
loses")

    idx, remove = find_winning_move(piles)
    if idx != -1:
        print(f"Winning move: take {remove} from pile {idx}")

    print("\n--- Full game simulation ---")
    play_nim(piles)

```

10. Code Explanation

Key parts of the code:

1. **Computing nim-sum:** XOR all pile sizes. `nim_sum ^= x` for each pile.
2. **Outcome check:** `nim_sum != 0` means first player wins.
3. **Finding winning move:** For each pile, check if `(pile_size ^ nim_sum) < pile_size`. If so, that pile can be reduced to `pile_size ^ nim_sum` to make the new XOR zero.
 - Why this condition works: XOR with a smaller number can either increase or decrease a number. We need the value to decrease (we can only remove stones, not add). The property is: `(x ^ nim_sum) < x` holds for at least one pile when `nim_sum != 0`.
4. **Simulation:** Alternates turns. At each turn, checks nim-sum, finds a winning move if available, updates piles.

Edge cases handled:

- All piles zero → `nim_sum = 0` → first player loses (correct, no stones to take).
- Single pile → `nim_sum = that pile` → always winning unless pile is 0.
- Large piles — XOR is $O(1)$ per pile regardless of size.

11. Complexity Analysis

Operation	Time	Space
Check if winning	$O(P)$ where P = number of piles	$O(1)$
Find winning move	$O(P)$	$O(1)$

Operation	Time	Space
Full simulation	$O(P \times \text{moves})$	$O(1)$

Overall: $O(P)$ per analysis. Space: $O(1)$.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Standard Nim	Multiple piles, remove any number from one pile	XOR of all piles	Nim Game (LeetCode 292 variant)
Nim with constraints on moves	Each pile has a limit (e.g., can remove at most k per turn)	Each pile becomes a take-away subgame; compute Grundy per pile	—
Misère Nim	Last move loses	XOR rule except when all piles are 1	GFG Misère Nim
Subtractive Nim	Remove from a subtraction set (not any number)	Grundy numbers per pile	—
Staircase Nim	Piles arranged on stairs, can move stones down	XOR of odd-indexed piles only	Staircase Nim (CF)

13. Common Mistakes

- Forgetting that XOR is the critical operation, not sum.
- Using sum instead of XOR → entirely wrong results.
- Not handling misère Nim correctly — the XOR rule reverses when all piles are size 1.
- Thinking the game is about taking "any number" within a range (that's take-away, not Nim).
- Trying to compute winning move by checking all possible removals instead of XOR formula.
- Integer overflow not an issue with XOR, but be careful with large pile sizes in Python (fine) or C++ (fine with `int`).

14. Edge Cases

- Single pile of size 0 → XOR = 0, first player loses.
- All piles size 1 with even count → XOR = 0, losing. Odd count → XOR = 1, winning (except misère).
- Single pile of size N → XOR = N , first player wins by taking all N stones.
- Two equal piles → XOR = 0, losing for first player.
- One massive pile and one small pile → XOR non-zero, check winning move.

15. Variations

Misère Nim

Last move loses. Standard XOR strategy works for all positions EXCEPT when all piles are of size 1. In that special case, the outcome is reversed: if the count of non-empty piles is odd, first player loses (instead of wins), and vice versa.

Staircase Nim

Piles are on stairs (index 0, 1, 2, ...). A move takes stones from a higher stair and moves them to a lower stair (or removes them from stair 0). Only XOR of piles on ODD-indexed stairs matters.

Turning Turtles / Wythoff's Game

Two piles. A player can either remove stones from one pile (like Nim) OR remove the same number from both piles. Uses Beatty sequences, not XOR.

16. Related Algorithms/Data Structures

Topic	Connection
Grundy Numbers	Each pile in Nim has Grundy number = pile size. XOR of Grundy numbers generalizes Nim to any impartial game.
Sprague-Grundy Theorem	Any impartial game is equivalent to a Nim heap of size equal to its Grundy number.
XOR (Bitwise Operations)	Core operation. Understanding XOR properties is essential for Nim.
Game Theory DP	For games that cannot be decomposed into independent piles.

17. Practice Problems

Easy

1. **Nim Game** — LeetCode 292. Single pile, take 1..3. Simple pattern.
2. **The Game of Nim** — LeetCode (biweekly). Standard XOR on multiple piles.

Medium

1. **Nim Game II** — LintCode / GFG. With upper limit on removal per pile.
2. **Stone Game IV** — LeetCode 1510. Take perfect squares. DP + win/lose.
3. **Misère Nim** — GFG / Codeforces. Implement misère version.

Hard

1. **Staircase Nim** — Codeforces (many variants). XOR odd-indexed piles.
2. **Nim with Candy** — CF Round. Multiple constraints on removal.
3. **Wythoff's Game** — POJ / UVa. Two piles, can remove from one or both equally.

18. Interview Explanation

"Nim is the fundamental impartial combinatorial game. Given multiple piles of stones, a player picks one pile and removes any positive number of stones. The player who takes the last stone wins. The key insight is the nim-sum: the XOR of all pile sizes. If the nim-sum is zero, the position is losing for the player to move. If non-zero, the player can make a move to make the nim-sum zero. Finding the winning move takes $O(P)$ time: for each pile, check if $(\text{pile_size} \oplus \text{nim_sum}) < \text{pile_size}$. Nim is important because the Sprague-Grundy theorem shows that every impartial game reduces to a Nim heap."

19. Revision Notes

- **Nim-sum = XOR of all pile sizes**
- **$\text{nim_sum} == 0 \rightarrow \text{losing}; \text{nim_sum} != 0 \rightarrow \text{winning}$**
- **Winning move:** pile where $(\text{size} \oplus \text{nim_sum}) < \text{size}$, reduce to $\text{size} \oplus \text{nim_sum}$
- **Misère:** XOR rule holds, except when all piles are size 1 (then reverse outcome)
- **Single pile:** always winning (take all)
- **Two equal piles:** XOR = 0, losing (mirror strategy)
- **$O(P)$ time, $O(1)$ space**

20. Final Cheat Sheet

Item	Detail
When to use	Multiple piles, remove any stones from one pile, last move wins
Main operation	Compute XOR of all pile sizes
Formula	$\text{nim_sum} = a_1 \oplus a_2 \oplus \dots \oplus a_n$
Code pattern	<pre>int nim = 0; for (int x : piles) nim ^= x;</pre>
Complexity	$O(P)$ time, $O(1)$ space
Winning move	<pre>if ((a[i] ^ nim_sum) < a[i]) -> remove a[i] - (a[i] ^ nim_sum)</pre>
Edge cases	All ones (misère), single pile = 0, two equal piles
Trap	Don't confuse XOR with sum; misère rule when all piles = 1

XOR STRATEGY

1. Overview

The XOR strategy is the application of bitwise XOR (exclusive OR) to solve combinatorial game theory problems, particularly those that decompose into independent subgames. It is most famous for solving Nim, but it also appears in many other CP and interview problems where XOR properties determine game outcomes or help construct optimal strategies.

2. Intuition

Simple explanation: XOR acts as a "balance checker" for impartial games. Each subgame contributes a value; XOR combines them. If the XOR of all contributions is zero, the overall position is "balanced" and losing for

the player to move. If non-zero, the player can unbalance it in their favor.

Analogy: Imagine a weighing scale where each subgame adds weights. XOR is like having a parity check across bit positions. If every bit column has an even number of 1s (XOR = 0), the position is a P-position. If any column has an odd count, the position is an N-position.

Step-by-step reasoning:

1. Each subgame state is assigned a number (its Grundy number or nim-value).
2. XOR these numbers together.
3. If XOR = 0, the combined position is losing.
4. If XOR $\neq$ 0, the combined position is winning, and there exists a move to make XOR = 0.

Why XOR and not sum or AND? XOR is the unique binary operation satisfying:

- Associativity: order doesn't matter.
- Identity: $x \wedge 0 = x$.
- Inverse: $x \wedge x = 0$.
- Cancellation: if $x \wedge y = x \wedge z$, then $y = z$.
- The ability to transition from non-zero to zero by modifying one component.

These properties match the structure of impartial combinatorial games perfectly.

3. When to Use It

- Any impartial game that decomposes into independent subgames
- Problems involving XOR of values and "can first player force a win?"
- Problems where each move changes one component's value
- Games where the terminal state has XOR = 0
- Problems about "balanced" or "unbalanced" states

Common trigger phrases: "XOR of", "bitwise XOR", "nim-sum", "Nim game", "Grundy numbers", "Sprague-Grundy", "take turns", "optimal play", "independent piles".

4. When Not to Use It

- Partizan games (players have different move sets) — XOR doesn't apply directly.
- Games with draws or cycles — XOR strategy assumes game terminates.
- Non-independent subgames (moves affect multiple subgames) — XOR breaks.
- Puzzles about maximum XOR pair/subarray (those are not game theory).
- When XOR is used only as a bit trick unrelated to game states.

5. Core Concepts

XOR Properties

- $x \wedge 0 = x$ (identity)
- $x \wedge x = 0$ (self-inverse)
- $x \wedge y = y \wedge x$ (commutative)
- $(x \wedge y) \wedge z = x \wedge (y \wedge z)$ (associative)

- $x \oplus x \oplus x \dots = x$ if odd count, 0 if even count

Nim-Sum

The XOR of all pile sizes (or Grundy numbers) in a game. The fundamental outcome determinant.

Balanced / Unbalanced Position

- **Balanced (P-position):** XOR = 0. Player to move loses with optimal play.
- **Unbalanced (N-position):** XOR $\neq$ 0. Player to move wins.

Winning Move Property

From a non-zero XOR, there is always at least one pile where $(\text{pile_size} \oplus \text{nim_sum}) < \text{pile_size}$. This allows reducing that pile to make XOR = 0.

Multi-Pile Generalization

If a game has independent piles with Grundy numbers $g_1, g_2, \dots, g_n$, the combined Grundy number is $g_1 \oplus g_2 \oplus \dots \oplus g_n$. Outcome is win if non-zero, lose if zero.

6. Step-by-Step Algorithm

1. **Identify independent components** of the game.
2. **Compute the Grundy number** (or value) for each component.
3. **XOR all component values** together.
4. **If XOR = 0:** Position is losing for the player to move.
5. **If XOR $\neq$ 0:** Position is winning. Find a component where moving can make XOR = 0:
 - For each component with value v , if $(v \oplus \text{total_xor}) < v$:
 - Transform this component to have value $v \oplus \text{total_xor}$.
 - This makes the new XOR = 0.
6. **Apply the move** and pass the turn.

7. Dry Run

Problem: Three piles = [3, 5, 6]. Compute outcome and winning move.

Component	Value
Pile 1	3 (binary 011)
Pile 2	5 (binary 101)
Pile 3	6 (binary 110)

Step 1: Compute XOR

```

3  = 011
5  = 101
6  = 110

```

```
-----
XOR = 000 (0)
```

Result: XOR = 0 → losing position. First player loses under optimal play.

Another example: Piles = [4, 5, 6]

```
4  = 100
5  = 101
6  = 110
-----
XOR = 111 (7)
```

Step 2: Find winning move:

- Pile 4: $4 \oplus 7 = 3$. Is $3 < 4$? Yes  → Reduce pile 4 to 3 (remove 1).
- New piles: [3, 5, 6]. XOR = $3 \oplus 5 \oplus 6 = 0$. First player has winning move.

Pile	Current	$v \oplus \text{XOR}$	New size	Valid?
4	4	3	3	 (3 < 4)
5	5	2	2	 (2 < 5)
6	6	1	1	 (1 < 6)

Any of these moves works. First player wins.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Compute XOR of a vector of values
int computeXOR(const vector<int>& values) {
    int xr = 0;
    for (int v : values) xr ^= v;
    return xr;
}

// Check if the combined position is winning (XOR != 0)
bool isWinningPosition(const vector<int>& values) {
    return computeXOR(values) != 0;
}

// Find a component index and its new value after a winning move
// Returns {component_index, new_value} if found, {-1, -1} otherwise
pair<int, int> findWinningMove(const vector<int>& values) {
    int total_xor = computeXOR(values);
```

```

    if (total_xor == 0) return {-1, -1};

    for (int i = 0; i < (int)values.size(); i++) {
        int new_val = values[i] ^ total_xor;
        if (new_val < values[i]) {
            return {i, new_val};
        }
    }
    return {-1, -1}; // not found (shouldn't happen)
}

// Generic function: given component values, determine outcome
string determineOutcome(const vector<int>& values) {
    int xr = computeXOR(values);
    if (xr == 0) return "Losing position (P-position)";
    else return "Winning position (N-position)";
}

int main() {
    vector<int> values = {4, 5, 6};

    cout << "Values: ";
    for (int v : values) cout << v << " ";
    cout << "\n";

    cout << "XOR = " << computeXOR(values) << "\n";
    cout << "Outcome: " << determineOutcome(values) << "\n";

    auto [idx, new_val] = findWinningMove(values);
    if (idx != -1) {
        cout << "Winning move: change component " << idx
              << " from " << values[idx] << " to " << new_val
              << " (remove " << values[idx] - new_val << ")\n";
    } else {
        cout << "No winning move available.\n";
    }

    return 0;
}

```

9. Python Implementation

```

from typing import List, Tuple

def compute_xor(values: List[int]) -> int:
    """Compute XOR of all values."""
    result = 0
    for v in values:
        result ^= v
    return result

```

```

def is_winning_position(values: List[int]) -> bool:
    return compute_xor(values) != 0

def find_winning_move(values: List[int]) -> Tuple[int, int]:
    """Return (index, new_value) for a winning move, or (-1, -1)."""
    total_xor = compute_xor(values)
    if total_xor == 0:
        return -1, -1

    for i, v in enumerate(values):
        new_val = v ^ total_xor
        if new_val < v:
            return i, new_val
    return -1, -1

def determine_outcome(values: List[int]) -> str:
    xr = compute_xor(values)
    return "Losing position (P-position)" if xr == 0 else "Winning position (N-position)"

if __name__ == "__main__":
    values = [4, 5, 6]
    print(f"Values: {values}")
    print(f"XOR = {compute_xor(values)}")
    print(f"Outcome: {determine_outcome(values)}")

    idx, new_val = find_winning_move(values)
    if idx != -1:
        print(f"Winning move: change component {idx} from {values[idx]} to {new_val}")
    else:
        print("No winning move available.")

```

10. Code Explanation

Key parts of the code:

1. **computeXOR**: Simple loop XORing all values. The core of the strategy.
2. **isWinningPosition**: Returns `true` if XOR $\neq 0$.
3. **findWinningMove**: For each component, checks if $(v \wedge total_xor) < v$.
 - This condition ensures we decrease the component's value (since we can only remove stones/units).
 - One such component always exists when $total_xor \neq 0$.
4. **determineOutcome**: Human-readable result.

Why $(v \wedge total_xor) < v$ works:

- Let `total_xor` be T . We want $\text{new_val} = v \oplus T$.
- After the move, $\text{new total XOR} = T \oplus v \oplus \text{new_val} = T \oplus v \oplus (v \oplus T) = 0$.
- But we need $\text{new_val} < v$ (we can only reduce the pile).
- Property: If $T \neq 0$, there is at least one v where $v \oplus T < v$. Specifically, look at the highest set bit in T ; any v with that bit set will satisfy $v \oplus T < v$.

11. Complexity Analysis

Operation	Time	Space
Compute XOR	$O(N)$	$O(1)$
Find winning move	$O(N)$	$O(1)$
Overall	$O(N)$	$O(1)$

Where N = number of components (piles).

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Standard XOR	Nim-like piles	XOR all values	Nim Game, Game of Nim
XOR with constraints	Each pile has removal limits	Compute Grundy per pile, XOR them	Multiple Grundy-based problems
XOR in chessboard	Placing pieces, independent rows/cols	XOR of values per row/col	CF Chessboard games
XOR subarray games	Take turns picking elements	XOR of elements in chosen sets	Various XOR games

13. Common Mistakes

- Using `sum` instead of `xor` — leads to wrong outcome.
- Forgetting that XOR must be computed on Grundy numbers, not original pile sizes, when there are move constraints.
- Applying XOR strategy to games where moves affect multiple piles.
- Not checking $\text{new_val} < v$ condition — the formula gives the target value, not the amount to remove.
- Integer type issues: XOR works on integers, but ensure 64-bit if values can be large.
- Assuming XOR strategy works for misère Nim without the special case.

14. Edge Cases

- All values zero $\rightarrow$ XOR = 0, losing position.
- Single non-zero value $\rightarrow$ XOR = value, winning.
- All equal values with even count $\rightarrow$ XOR = 0, losing.
- All equal values with odd count $\rightarrow$ XOR = value, winning.
- Values large (up to 2^{63}) — use `long long` in C++, Python handles big ints.

15. Variations

XOR in Combinatorial Games Beyond Nim

The XOR strategy is the engine behind the Sprague-Grundy theorem. Any impartial game's Grundy number is the mex of Grundy numbers of reachable states. XOR combines independent components.

XOR of Prefix/Suffix

Sometimes used in non-game problems like "can we partition an array such that XOR of segments equals something." Not game theory but uses the same XOR tool.

XOR Linked List

A linked list where each node stores XOR of prev and next addresses. Not related to game theory.

16. Related Algorithms/Data Structures

Topic	Connection
Nim Game	The canonical use of XOR strategy.
Grundy Numbers	Each state's Grundy number is combined via XOR.
Sprague-Grundy Theorem	Proves XOR is the correct composition operator.
Bit Manipulation	XOR is a bitwise operation; understanding binary helps.
Bouton's Theorem	The original proof that XOR solves Nim.

17. Practice Problems

Easy

1. **XOR Operation in an Array** — LeetCode 1486. Simple XOR usage (not game theory but good to learn XOR).
2. **Nim Game** — LeetCode 292. Basic understanding.

Medium

1. **Game of Nim** — LeetCode (various). Multiple piles, XOR strategy.
2. **XOR Game** — GFG. Given array, players take turns picking elements, XOR of selected determines winner.

Hard

1. **Cat and Mouse** — LeetCode 913. Graph game, uses Grundy + XOR.
2. **CF Round: Nim with Constraints** — Many Codeforces problems combine XOR with DP or combinatorial constraints.

18. Interview Explanation

"The XOR strategy is how we determine the winner in impartial combinatorial games with independent components. We compute the XOR of all component values. If the XOR is zero, the position is losing for the player to move. If non-zero, it's winning. From a non-zero XOR, there is always a move that makes the XOR zero, by adjusting one component. This works because XOR has the inverse property: $x \oplus x = 0$. The Sprague-Grundy theorem proves this is the universal strategy for all impartial games."

19. Revision Notes

- **XOR = 0** → losing (P-position)
- **XOR ≠ 0** → winning (N-position)
- **Winning move condition:** $(v \oplus \text{total_xor}) < v \rightarrow$ reduce v to $v \oplus \text{total_xor}$
- **Always one such component** when XOR ≠ 0
- **Key property:** XOR is its own inverse
- **Complexity:** O(N) time, O(1) space

20. Final Cheat Sheet

Item	Detail
When to use	Independent subgames, each contributes a value
Main operation	XOR of all component values
Formula	$\text{total_xor} = v_1 \oplus v_2 \oplus \dots \oplus v_n$
Code pattern	<code>int xr = accumulate(v.begin(), v.end(), 0, bit_xor<>())</code>
Complexity	O(N) time, O(1) space
Winning condition	$\text{total_xor} \neq 0$
Winning move	$\text{new_val} = v[i] \oplus \text{total_xor}$ where $\text{new_val} < v[i]$
Edge cases	All zeros, single value, large values up to 2^{63}

GRUNDY NUMBERS

1. Overview

Grundy numbers (also called nimbers) assign a non-negative integer to each state of an impartial combinatorial game. The Grundy number represents the state's "value" — specifically, the size of the equivalent Nim heap. The key rule: from a state with Grundy number g , you can move to any state with a Grundy number less than g , and the mex (minimum excluded) of reachable Grundy numbers determines g .

2. Intuition

Simple explanation: Every state in an impartial game behaves like a Nim heap of a certain size. The Grundy number tells you that size. If you know the Grundy number of each state, you only need to know Nim to solve the game.

Analogy: Think of each game position as a pile of stones in Nim, but the "pile size" is not obvious — it's computed via mex. The mex rule is like asking: "What is the smallest non-negative integer that I cannot reach in one move?"

Step-by-step reasoning:

1. Terminal states have no moves. They can reach $\{ \}$ (empty set of Grundy numbers). The smallest non-negative integer NOT in $\{ \}$ is 0. So $\text{Grundy}(\text{terminal}) = 0$.
2. For any state, compute Grundy numbers of all reachable states. Collect them in a set.
3. Find the mex (minimum excludant) — the smallest non-negative integer not in the set.
4. That mex is the Grundy number of the current state.

Why it works: Grundy numbers transform any impartial game into Nim. The mex operation preserves the game structure: from a position with Grundy g , you can move to any position with Grundy $h < g$ (just like removing stones in Nim). The Sprague-Grundy theorem proves this equivalence.

3. When to Use It

- Impartial games with constraints on moves (can't remove any number, only specific amounts)
- Breaking a complex game into independent subgames and combining results
- Any impartial game where the state space is manageable ($\leq 10^5$ — 10^6 states)
- Games on DAGs (directed acyclic graphs) where states don't repeat
- Problems asking "can first player win?" with complex move rules

Common trigger phrases: "Grundy numbers", "mex", "nimbers", "impartial game", "optimal play on a DAG", "independent subgames", "each move on one pile", "compute the winner".

4. When Not to Use It

- State space too large for DP (use pattern-finding or mathematical formulas)
- Partizan games (chess, checkers — different moves for each player)
- Games with draws or cycles
- Games where the same state can repeat (non-DAG) — Grundy assumes game terminates
- Problems with simple patterns that don't need Grundy (e.g., basic Nim or take-away)
- Misère games (Grundy theory changes significantly in misère)

5. Core Concepts

Grundy Number (Nimber)

A non-negative integer assigned to a game state. Terminal states have Grundy = 0. The Grundy number represents the state's equivalence to a Nim heap.

Mex (Minimum Excludant)

For a set S of non-negative integers, $\text{mex}(S)$ = smallest non-negative integer NOT in S .

- $\text{mex}(\{0, 1, 3\}) = 2$
- $\text{mex}(\{0, 1, 2, 3\}) = 4$
- $\text{mex}(\{ \}) = 0$

Recursive Definition

```
G(state) = mex({ G(next_state) for all reachable next_state })
```

Terminal State

State with no outgoing moves. Grundy = 0.

Winning/Losing via Grundy

- Grundy = 0 → losing state (P-position)
- Grundy ≠ 0 → winning state (N-position)

Nim-Sum of Grundy Numbers

For composite games (multiple independent piles): XOR all Grundy numbers. Result:

- XOR = 0 → losing
- XOR ≠ 0 → winning

6. Step-by-Step Algorithm

1. **Model the game:** Represent states as nodes in a DAG. Moves are edges.
2. **Identify terminal states:** States with no outgoing edges. Assign Grundy = 0.
3. **Topological ordering:** Process states in reverse topological order (or use DFS + memoization).
4. **For each state:**
 - Compute Grundy numbers of all reachable next states.
 - Collect them in a set.
 - Compute mex of that set.
 - Assign mex as the state's Grundy number.
5. **For composite games** (multiple piles):
 - Compute Grundy number for each pile independently.
 - XOR all Grundy numbers.
 - If XOR = 0 → losing for first player.
 - If XOR ≠ 0 → winning for first player.
6. **Find winning move (optional):**
 - XOR all Grundy numbers → G_{total} .
 - For each pile with Grundy g_i : if $(g_i \oplus G_{total}) < g_i$:
 - Need to reduce this pile's Grundy to $g_i \oplus G_{total}$.
 - Find a move in that pile's state space that achieves that Grundy value.

7. Dry Run

Problem: Take-away game from a single pile of n stones. You can take 1, 3, or 4 stones per move. Compute Grundy numbers for $n = 0$ to 10.

n	Reachable Grundy Set	mex	Grundy(n)
0	{}	0	0
1	{G(0)} = {0}	1	1
2	{G(1)} = {1}	0	0
3	{G(2), G(0)} = {0, 0} = {0}	1	1
4	{G(3), G(1), G(0)} = {1, 1, 0} = {0, 1}	2	2
5	{G(4), G(2), G(1)} = {2, 0, 1} = {0, 1, 2}	3	3
6	{G(5), G(3), G(2)} = {3, 1, 0} = {0, 1, 3}	2	2
7	{G(6), G(4), G(3)} = {2, 2, 1} = {1, 2}	0	0
8	{G(7), G(5), G(4)} = {0, 3, 2} = {0, 2, 3}	1	1
9	{G(8), G(6), G(5)} = {1, 2, 3} = {1, 2, 3}	0	0
10	{G(9), G(7), G(6)} = {0, 0, 2} = {0, 2}	1	1

So Grundy(10) = 1 $\neq$ 0 $\rightarrow$ winning for first player (n=10 with move set {1, 3, 4}).

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Compute Grundy numbers for a take-away game
// moves: vector of allowed stone removals
// n: maximum pile size to compute
vector<int> computeGrundy(int n, const vector<int>& moves) {
    vector<int> grundy(n + 1, 0);

    // Terminal state: grundy[0] = 0 (already set)
    for (int i = 1; i <= n; i++) {
        set<int> reachable;
        for (int m : moves) {
            if (m <= i) {
                reachable.insert(grundy[i - m]);
            }
        }
        // Compute mex
        int mex = 0;
        while (reachable.count(mex)) mex++;
        grundy[i] = mex;
    }

    return grundy;
}

// Compute mex of a set of integers (alternative O(S) version)
```

```

int computeMex(const unordered_set<int>& s) {
    int mex = 0;
    while (s.count(mex)) mex++;
    return mex;
}

// For a composite game with multiple independent piles,
// compute the Grundy number of the combined game
int compositeGrundy(const vector<int>& pileGrundy) {
    int xr = 0;
    for (int g : pileGrundy) xr ^= g;
    return xr;
}

// Determine if a set of pile Grundy numbers is winning
bool isWinningComposite(const vector<int>& pileGrundy) {
    return compositeGrundy(pileGrundy) != 0;
}

int main() {
    // Take-away game: remove 1, 3, or 4 stones
    vector<int> moves = {1, 3, 4};
    int maxN = 20;

    vector<int> grundy = computeGrundy(maxN, moves);

    cout << "Grundy numbers for moves {1, 3, 4}:\n";
    for (int i = 0; i <= maxN; i++) {
        cout << "G(" << i << ") = " << grundy[i] << "\n";
    }

    // Example: two piles of sizes 5 and 7
    vector<int> piles = {5, 7};
    vector<int> pileGrundy = {grundy[5], grundy[7]};

    cout << "\nPiles: ";
    for (int p : piles) cout << p << " ";
    cout << "\nGrundy numbers: ";
    for (int g : pileGrundy) cout << g << " ";
    cout << "\n";
    cout << "Combined XOR: " << compositeGrundy(pileGrundy) << "\n";
    cout << "Outcome: " << (isWinningComposite(pileGrundy) ? "Winning" : "Losing")
    << "\n";

    return 0;
}

```

9. Python Implementation

```

from typing import List, Set

```

```

def compute_grundy(n: int, moves: List[int]) -> List[int]:
    """Compute Grundy numbers for a take-away game up to n stones."""
    grundy = [0] * (n + 1)

    for i in range(1, n + 1):
        reachable = set()
        for m in moves:
            if m <= i:
                reachable.add(grundy[i - m])
        # Compute mex
        mex = 0
        while mex in reachable:
            mex += 1
        grundy[i] = mex

    return grundy

def compute_mex(s: Set[int]) -> int:
    """Compute minimum excludant of a set."""
    mex = 0
    while mex in s:
        mex += 1
    return mex

def composite_grundy(pile_grundy: List[int]) -> int:
    """XOR of all pile Grundy numbers."""
    result = 0
    for g in pile_grundy:
        result ^= g
    return result

def is_winning_composite(pile_grundy: List[int]) -> bool:
    return composite_grundy(pile_grundy) != 0

if __name__ == "__main__":
    moves = [1, 3, 4]
    max_n = 20

    grundy = compute_grundy(max_n, moves)

    print(f"Grundy numbers for moves {moves}:")
    for i in range(max_n + 1):
        print(f"G({i}) = {grundy[i]}")

    # Two piles of sizes 5 and 7
    piles = [5, 7]
    pile_grundy = [grundy[5], grundy[7]]

    print(f"\nPiles: {piles}")

```

```
print(f"Grundy numbers: {pile_grundy}")
print(f"Combined XOR: {composite_grundy(pile_grundy)}")
print(f"Outcome: {'Winning' if is_winning_composite(pile_grundy) else 'Losing'}")
```

10. Code Explanation

Key parts of the code:

1. **computeGrundy**: Iterates from 0 to n (bottom-up DP).
 - For each state **i**, collect Grundy numbers of all states reachable by subtracting each valid move.
 - Use a **set** to deduplicate reachable Grundy numbers.
 - Find mex by starting from 0 and incrementing while the number is in the set.
 - Assign mex to **grundy[i]**.
2. **computeMex**: Alternative standalone mex function using `unordered_set` for efficiency.
3. **compositeGrundy**: XOR all pile Grundy numbers to get the combined game Grundy.
4. **isWinningComposite**: Non-zero XOR means winning.

Edge cases handled:

- **n = 0**: `grundy[0] = 0` (correct, terminal).
- Move larger than current pile: skipped via **m <= i** check.
- No moves from a state: reachable set is empty, `mex({}) = 0`.

11. Complexity Analysis

Case	Time	Space
Single state Grundy	$O(M \times \log M)$ where M = number of moves	$O(M)$ for set
All states up to N	$O(N \times M \times \log M)$	$O(N + M)$
With DP + set	$O(N \times M \times \log M)$	$O(N)$
Optimized (array instead of set)	$O(N \times M + N \times \text{maxGrundy})$	$O(N + \text{maxGrundy})$

Best case: Pattern emerges quickly (periodic Grundy) $\rightarrow O(N)$ to detect pattern. **Worst case:** States up to 10^6 with many moves.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Subtraction Game	Take from set S	Compute Grundy per pile, XOR them	CF: Subtraction Game

Pattern	How to Identify	Approach	Example Problems
DAG Game	Move through graph nodes	DFS + memoization of Grundy	CF: Game on a DAG
Board Games	Pieces on grid, independent rows	Compute Grundy per row/column	Chessboard games
Partition Game	Split a number into parts	Each partition is a subgame with its own Grundy	CF: Split and XOR
Green Hackenbush	Cutting edges from a tree	Grundy numbers for rooted trees	Tree-cutting games

13. Common Mistakes

- Computing mex incorrectly (not checking from 0).
- Using `int` set for reachable Grundy numbers when max Grundy is large — use `vector<bool>` for speed.
- Forgetting that two different states can have the same Grundy number — set deduplication is essential.
- Not realizing Grundy depends on the move set, not just pile size.
- Applying Grundy to games with cycles (Grundy assumes DAG).
- Trying to compute Grundy for partizan games (use other theory).
- Thinking Grundy = pile size always — that's only true for Nim (remove any number).

14. Edge Cases

- Terminal state (no moves) → Grundy = 0.
- State with only self-loop → not a DAG, Grundy undefined.
- State where all Grundy numbers up to K are reachable → Grundy = $K + 1$.
- State where Grundy number can be arbitrarily large → check if bound exists.
- Duplicate Grundy numbers in reachable set — use set to deduplicate.
- Very large move set — may slow DP significantly.

15. Variations

Grundy on Directed Acyclic Graphs

The most general form. Each node is a state. Compute Grundy using DFS + memoization over the graph.

Periodic Grundy

Many take-away games have periodic Grundy sequences. Useful for large N where DP is impossible.

Grundy for Decoupled Games

When moves don't interact (Nim piles), Grundy per component is computed independently and XORed.

Grundy with Symmetry

Some games have symmetric states — compute once and reuse.

16. Related Algorithms/Data Structures

Topic	Connection
Sprague-Grundy Theorem	The theoretical foundation.
Nim Game	Nim is the special case where $\text{Grundy}(\text{pile}) = \text{pile size}$.
XOR Strategy	How Grundy numbers are combined.
Game Theory DP	Computing Grundy via DP on state space.
Mex (Data Structure)	Efficient mex computation using segment trees or binary indexed trees.

17. Practice Problems

Easy

1. **Nim Game** — LeetCode 292. Simple case (but also Grundy: $G(n) = n \% 4$ for $k=3$).
2. **Stone Game** — LeetCode 877. Even piles, pick from ends.

Medium

1. **Divisor Game** — LeetCode 1025. Alice and Bob pick divisors. Grundy-like DP.
2. **Game on a Graph** — CF Round. DAG game with Grundy computation.
3. **Subtraction Game** — SPOJ. Classic Grundy problem.

Hard

1. **Cat and Mouse** — LeetCode 913. Graph game with Grundy-like analysis.
2. **Green Hackenbush** — Codeforces / AtCoder. Tree-cutting game.
3. **Impartial Game** — CF Round (various). Complex move rules, compute Grundy via DP + mex.

18. Interview Explanation

"Grundy numbers are a way to assign a numerical value to each state in an impartial combinatorial game. The Grundy number of a terminal state is 0. For any state, we compute the set of Grundy numbers of all states reachable in one move. The Grundy number of the current state is the mex — the minimum excluded non-negative integer — of that set. A state is losing if its Grundy number is 0, winning otherwise. When a game consists of independent subgames, we XOR their Grundy numbers to get the combined Grundy. This is the Sprague-Grundy theorem in action."

19. Revision Notes

- **$G(\text{state}) = \text{mex}(\{G(\text{next_state}) \text{ for all moves}\})$**
- **$G(\text{terminal}) = 0$**
- **$G = 0 \rightarrow \text{losing}; G \neq 0 \rightarrow \text{winning}$**
- **Composite game:** XOR of all component Grundy numbers
- **Mex:** smallest non-negative integer not in set

- **DP**: bottom-up or DFS + memoization
- **Periodic pattern**: many games have periodic Grundy sequences

20. Final Cheat Sheet

Item	Detail
When to use	Impartial games with move constraints, on DAG
Main operation	mex of reachable Grundy numbers
Formula	$G(s) = \text{mex}(\{G(t) \mid s \rightarrow t\})$
Code pattern	<pre>set<int> reachable; for move: reachable.insert(grundy[i-move]); while(reachable.count(mex)) mex++;</pre>
Complexity	$O(N \times M \times \log M)$ for DP
Winning condition	$G(\text{state}) \neq 0$ (single) or $\text{XOR} \neq 0$ (composite)
Edge cases	Terminal $\rightarrow 0$, no reachable states $\rightarrow 0$
Trap	Don't forget deduplication; don't apply to cyclic games

SPRAGUE-GRUNDY THEOREM

1. Overview

The Sprague-Grundy theorem is the central result in combinatorial game theory. It states that every impartial combinatorial game under normal play is equivalent to a Nim heap of a certain size (its Grundy number). Furthermore, the disjunctive sum (playing multiple independent games in parallel, where a move is made in exactly one component) has a combined Grundy number equal to the XOR of the component Grundy numbers.

2. Intuition

Simple explanation: Any impartial game — no matter how complex its rules — behaves exactly like a pile of stones in Nim. The size of that equivalent Nim pile is the Grundy number. And when you play multiple games at once, it's just like playing Nim with multiple piles.

Analogy: Think of every impartial game as a "black box" with a "Nim-equivalence dial." The Sprague-Grundy theorem says you can turn the dial to some number, throw away the black box, and just play Nim with that number. The outcome under optimal play is identical.

Step-by-step reasoning:

1. Every impartial game state has a Grundy number (defined via mex).
2. A position with Grundy number g is equivalent to a Nim heap of size g .
3. If you have multiple independent games (you move in one, I move in one), the whole situation is equivalent to Nim with heap sizes = Grundy numbers of each component.

4. The XOR of those Grundy numbers determines the outcome: zero = losing, non-zero = winning.

Why it works: The mex operation preserves the structural properties of Nim. From a Nim heap of size g , you can move to any heap of size h where $0 \leq h < g$. From a game state with Grundy g , you can move to any state whose Grundy is $h < g$ (by definition of mex). The equivalence is exact.

3. When to Use It

- Any game where states decompose into independent subgames
- Proving that XOR strategy works for any impartial game
- Converting unfamiliar games into Nim equivalents
- Problems where a game has multiple "piles" with complex move rules per pile
- Analyzing impartial games in general

Common trigger phrases: "Sprague-Grundy theorem", "reduce to Nim", "impartial combinatorial game", "disjunctive sum", "independent components", "Grundy numbers", "nimbers".

4. When Not to Use It

- Partizan games (each player has different moves) — use Conway's theory or Minimax
- Games with draws or cycles
- Misère games (the theorem fails in misère play for most games)
- Games where components are not independent (moves affect multiple components)
- When you need the actual winning strategy, not just the outcome (the theorem tells you the outcome, finding the move may still require exploring component state spaces)

5. Core Concepts

Impartial Game

A game where both players have the same set of moves from every position. No hidden information, no chance.

Disjunctive Sum

Playing multiple games in parallel. On your turn, you choose exactly one component and make a move in it. The game ends when no moves are possible in any component.

Grundy Number (Nimber)

The Grundy number of a state is the mex of Grundy numbers of its options. Terminal state has Grundy 0.

Nim Equivalence

A game state with Grundy g is equivalent to a Nim heap of size g . Players who know Grundy can treat any state as a Nim heap.

XOR Composition

The Grundy number of the disjunctive sum of games $G_1, G_2, \dots, G_n$ is:

$$G(G_1 + G_2 + \dots + G_n) = G(G_1) \wedge G(G_2) \wedge \dots \wedge G(G_n)$$

Outcome via XOR

- XOR of component Grundy numbers = 0 → losing for player to move.
- XOR ≠ 0 → winning for player to move.

6. Step-by-Step Algorithm

1. **Decompose** the game into independent components (e.g., individual Nim piles, or separate subgames that don't interact).
2. **For each component**, compute its Grundy number:
 - Identify terminal states (assign 0).
 - For each state, compute mex of Grundy numbers of reachable states.
 - Use DP or DFS + memoization.
3. **XOR** all component Grundy numbers: $\text{total_grundy} = g_1 \wedge g_2 \wedge \dots \wedge g_n$.
4. **Determine outcome:**
 - $\text{total_grundy} == 0 \rightarrow$ P-position (player to move loses)
 - $\text{total_grundy} != 0 \rightarrow$ N-position (player to move wins)
5. **(Optional) Find winning move:**
 - For each component with Grundy g_i :
 - If $(g_i \wedge \text{total_grundy}) < g_i$:
 - Search within that component for a state with Grundy $g_i \wedge \text{total_grundy}$.
 - Make the move to that state.

7. Dry Run

Problem: Two independent take-away piles:

- Pile 1: 5 stones, can remove 1 or 2 stones.
- Pile 2: 4 stones, can remove 1, 3, or 4 stones.

Step 1: Compute Grundy numbers for each pile's move set.

For pile 1 (moves {1, 2}):

n	Reachable	mex	Grundy
0	{}	0	0
1	{0}	1	1
2	{1, 0} = {0, 1}	2	2
3	{2, 1} = {1, 2}	0	0
4	{0, 2} = {0, 2}	1	1
5	{1, 0} = {0, 1}	2	2

So $G1 = \text{Grundy}(\text{pile1 with 5 stones}) = 2$.

For pile 2 (moves {1, 3, 4}):

n	Reachable	mex	Grundy
0	{}	0	0
1	{0}	1	1
2	{1}	0	0
3	{0, 0} = {0}	1	1
4	{1, 0, 0} = {0, 1}	2	2

$G2 = \text{Grundy}(\text{pile2 with 4 stones}) = 2$.

Step 2: $\text{XOR} = 2 \wedge 2 = 0$.

Result: $\text{XOR} = 0 \rightarrow$ losing position. First player loses under optimal play.

Winning move check: Since $\text{XOR} = 0$, no winning move exists.

(If we changed pile 2 to size 3: $G2 = 1$, $\text{XOR} = 2 \wedge 1 = 3 \neq 0 \rightarrow$ winning.)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Compute Grundy numbers for a game with given move set
vector<int> computeGrundy(int maxN, const vector<int>& moves) {
    vector<int> grundy(maxN + 1, 0);

    for (int i = 1; i <= maxN; i++) {
        vector<bool> seen(grundy.size() + 1, false);
        for (int m : moves) {
            if (m <= i) {
                seen[grundy[i - m]] = true;
            }
        }
        // mex
        int mex = 0;
        while (mex < (int)seen.size() && seen[mex]) mex++;
        grundy[i] = mex;
    }

    return grundy;
}

// XOR of all Grundy numbers (Sprague-Grundy combined)
int combineGrundy(const vector<int>& grundyNumbers) {
    int xr = 0;
    for (int g : grundyNumbers) xr ^= g;
    return xr;
}
```

```

    for (int g : grundyNumbers) xr ^= g;
    return xr;
}

// Determine if the combined position is winning
bool isWinning(const vector<int>& grundyNumbers) {
    return combineGrundy(grundyNumbers) != 0;
}

// Struct to represent a composite impartial game
struct SpragueGrundyGame {
    vector<vector<int>> componentStates; // each component has its own states
    vector<vector<int>> componentMoves; // moves per component
    vector<int> currentIndices; // current state index for each component

    // Compute Grundy numbers for a component with given max states and moves
    vector<int> computeComponentGrundy(int maxStates, const vector<int>& moves) {
        return computeGrundy(maxStates, moves);
    }

    // Check if current combined position is winning
    bool isCurrentWinning(const vector<vector<int>>& componentGrundy) {
        int total = 0;
        for (size_t i = 0; i < currentIndices.size(); i++) {
            total ^= componentGrundy[i][currentIndices[i]];
        }
        return total != 0;
    }
};

int main() {
    // Example: pile 1: 5 stones, moves {1, 2}
    //           pile 2: 4 stones, moves {1, 3, 4}

    auto grundy1 = computeGrundy(5, {1, 2});
    auto grundy2 = computeGrundy(4, {1, 3, 4});

    cout << "Pile 1 Grundy: " << grundy1[5] << " (n=5, moves {1,2})\n";
    cout << "Pile 2 Grundy: " << grundy2[4] << " (n=4, moves {1,3,4})\n";

    vector<int> stateGrundy = {grundy1[5], grundy2[4]};
    int total = combineGrundy(stateGrundy);

    cout << "Combined XOR (Sprague-Grundy): " << total << "\n";
    cout << "Outcome: " << (total == 0 ? "Losing" : "Winning") << "\n";

    return 0;
}

```

9. Python Implementation

```

from typing import List

def compute_grundy(max_n: int, moves: List[int]) -> List[int]:
    """Compute Grundy numbers for a game with given move set."""
    grundy = [0] * (max_n + 1)

    for i in range(1, max_n + 1):
        seen = [False] * (len(grundy) + 1)
        for m in moves:
            if m <= i:
                seen[grundy[i - m]] = True
        # Mex
        mex = 0
        while mex < len(seen) and seen[mex]:
            mex += 1
        grundy[i] = mex

    return grundy

def combine_grundy(grundy_numbers: List[int]) -> int:
    """XOR of all Grundy numbers (Sprague-Grundy theorem)."""
    result = 0
    for g in grundy_numbers:
        result ^= g
    return result

def is_winning(grundy_numbers: List[int]) -> bool:
    return combine_grundy(grundy_numbers) != 0

class SpragueGrundyGame:
    """Composite impartial game using Sprague-Grundy theorem."""

    def __init__(self, component_sizes: List[int], moves_list: List[List[int]]):
        self.component_sizes = component_sizes
        self.moves_list = moves_list
        self.component_grundy = []

        for size, moves in zip(component_sizes, moves_list):
            self.component_grundy.append(compute_grundy(size, moves))

    def current_outcome(self, state_indices: List[int]) -> str:
        total = 0
        for i, idx in enumerate(state_indices):
            total ^= self.component_grundy[i][idx]
        return "Losing" if total == 0 else "Winning"

if __name__ == "__main__":

```

```
grundy1 = compute_grundy(5, [1, 2])
grundy2 = compute_grundy(4, [1, 3, 4])

print(f"Pile 1 Grundy: {grundy1[5]} (n=5, moves [1,2])")
print(f"Pile 2 Grundy: {grundy2[4]} (n=4, moves [1,3,4])")

state_grundy = [grundy1[5], grundy2[4]]
total = combine_grundy(state_grundy)

print(f"Combined XOR (Sprague-Grundy): {total}")
print(f"Outcome: {'Losing' if total == 0 else 'Winning'})")
```

10. Code Explanation

Key parts of the code:

- 1. **computeGrundy**: Bottom-up DP to compute Grundy numbers for a single component. Uses a `vector<bool> seen` for O(1) lookup instead of `set` for better performance when max Grundy is bounded.
- 2. **combineGrundy**: XOR of all component Grundy numbers. This is the direct application of the Sprague-Grundy theorem.
- 3. **isWinning**: Returns true if XOR $\neq$ 0.
- 4. **SpragueGrundyGame class**: Encapsulates a composite game with multiple components, each with its own move rules.

Why the theorem matters in code: Instead of computing the outcome of the combined game directly (which could have enormous state space), we decompose, compute Grundy per component independently, and XOR. This is exponentially faster.

11. Complexity Analysis

Operation	Time	Space
Compute Grundy for one component (N states, M moves)	$O(N \times M + N \times \text{maxG})$	$O(N + \text{maxG})$
Combine via XOR	$O(K)$ where K = components	$O(1)$
Find winning move in composite game	$O(K \times S)$ where S = search per component	$O(1)$

Overall: $O(\sum(N_i \times M_i) + K)$ where N_i, M_i are states and moves per component.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Standard SG	Multiple independent piles with same/different move rules	Compute Grundy per pile, XOR	Various CF problems
SG on Graph	Game state is a position on a DAG	DFS + memoization for Grundy	CF: Game on DAG
SG with Board	Pieces on board, independent columns/rows	Grundy per row, XOR	Chessboard impartial games
SG with Tree	Cutting edges from trees	Grundy for rooted trees	Green Hackenbush
SG with Partition	Splitting number into sum of numbers	Grundy of each partition	Split games

13. Common Mistakes

- Forgetting to compute Grundy per component: using pile size directly when moves are restricted.
- Applying SG to partizan games.
- Applying SG to misère games without special handling.
- Not handling the case where multiple components share state dependencies (not truly independent).
- Assuming XOR of Grundy numbers directly gives the outcome without checking if the game is normal-play.
- Forgetting that the theorem applies to the disjunctive sum (move in exactly one component per turn). If you can move in multiple components at once, SG doesn't apply.

14. Edge Cases

- All components terminal $\rightarrow$ all Grundy = 0, XOR = 0 $\rightarrow$ losing.
- One component has Grundy 0, others non-zero $\rightarrow$ XOR may be non-zero $\rightarrow$ winning.
- Very large component state space $\rightarrow$ need pattern detection instead of full DP.
- Components with periodic Grundy sequences $\rightarrow$ compute pattern up to period, not full DP.
- Multiple identical components — XOR of even count = 0, odd count = value.

15. Variations

Sprague-Grundy for Misère

In misère play (last move loses), the theorem does not generally hold. However, for Nim specifically, a modified rule exists (reverse outcome when all piles are size 1).

SG for Green Hackenbush

Each edge of a rooted tree is a component. Grundy number of a tree is the XOR of Grundy numbers of branches, computed via the "colon principle."

SG with Symmetry

Exploit symmetry to reduce computation. Equivalent states have the same Grundy number.

16. Related Algorithms/Data Structures

Topic	Connection
Grundy Numbers	The values computed for each state.
Nim Game	The canonical equivalent game.
XOR Strategy	How the theorem combines components.
Game Theory DP	Computing Grundy numbers efficiently.
Mex	The operation at the heart of Grundy computation.
Minimax	For partizan games where SG doesn't apply.

17. Practice Problems

Easy

1. **Nim Game** — LeetCode 292. Application of Nim equivalence.
2. **Game of Nim** — LeetCode (various). Multiple piles, SG applies directly.

Medium

1. **Stone Game IV** — LeetCode 1510. Take perfect squares. Compute Grundy.
2. **Can I Win** — LeetCode 464. Numbers 1..max, reach target. Grundy-like DP.
3. **Game on a Graph** — CF Round. DAG + Grundy computation.

Hard

1. **Cat and Mouse** — LeetCode 913. Graph game requiring SG-like analysis.
2. **Green Hackenbush** — Codeforces. Tree-cutting with Grundy.
3. **CF Round: SG with constraints** — Various CF problems combining SG with combinatorial analysis.

18. Interview Explanation

"The Sprague-Grundy theorem is the central result in impartial combinatorial game theory. It states that every impartial game position is equivalent to a Nim heap, where the heap size is the Grundy number of the position. The Grundy number is computed recursively: it's the mex of the Grundy numbers of all reachable positions. When playing multiple independent games, the combined Grundy number is the XOR of the individual Grundy numbers. If the XOR is zero, the position is losing; if non-zero, it's winning. This lets us analyze complex games by decomposing them into independent components and using XOR to combine results."

19. Revision Notes

- **Every impartial game state $\equiv$ Nim heap of size = Grundy number**
- **Composite game:** XOR of Grundy numbers of components
- **XOR = 0 $\rightarrow$ losing; XOR $\neq$ 0 $\rightarrow$ winning**
- **Grundy = mex of reachable Grundy numbers**
- **Only for normal play** (last move wins)

- **Only for disjunctive sum** (move in one component per turn)
- **Complexity:** $O(\text{total_states} \times \text{moves_per_state})$ to compute all Grundy numbers

20. Final Cheat Sheet

Item	Detail
When to use	Any impartial game with independent components
Main operation	XOR of Grundy numbers of all components
Theorem	$G(\Sigma \text{ components}) = G_1 \wedge G_2 \wedge \dots \wedge G_n$
Code pattern	<pre>int total = 0; for (int g : grundy) total ^= g;</pre>
Complexity	Depends on Grundy computation per component
Outcome	$\text{XOR} \neq 0 \rightarrow \text{winning}; \text{XOR} = 0 \rightarrow \text{losing}$
Constraints	Normal play, impartial, disjunctive sum, DAG
Trap	Doesn't work for misère, partizan, or cyclic games

MINIMAX

1. Overview

Minimax is a decision-making algorithm for two-player turn-based games with opposing goals. One player (Maximizer) tries to maximize the score, and the other (Minimizer) tries to minimize it. Minimax explores the game tree to find the optimal move for the current player, assuming the opponent also plays optimally.

2. Intuition

Simple explanation: Imagine both players are trying to achieve opposite goals. You (Max) want the highest score; the opponent (Min) wants the lowest. Minimax says: on your turn, pick the move that gives you the best possible worst-case outcome — the move that maximizes the minimum you can guarantee.

Analogy: Think of a negotiation where you want the highest price and the buyer wants the lowest. You propose a price; they counter-propose. Minimax simulates this back-and-forth and finds the price you can guarantee regardless of their response.

Step-by-step reasoning:

1. The game has a finite tree of states.
2. Leaf nodes have a score (evaluation).
3. On Max's turn: choose the child with the highest score.
4. On Min's turn: choose the child with the lowest score.
5. Propagate these choices up the tree to the root.

Why it works: Minimax computes the value of the game under optimal play. If both players act optimally, the actual outcome will be exactly the minimax value of the root. Any deviation by the opponent only improves

the outcome for the player who deviated.

3. When to Use It

- Two-player zero-sum games (one player's gain is the other's loss)
- Perfect information games
- Game tree is small enough to explore (or can be pruned)
- Problems asking "optimal score with both players playing optimally"
- Evaluating game states with a heuristic function
- Tic-tac-toe, Connect Four, checkers endgames, small-board chess puzzles

Common trigger phrases: "optimal play", "maximize your score while minimizing opponent's", "both players play optimally", "difference in scores", "evaluation function", "game tree".

4. When Not to Use It

- Game tree is too large (use Alpha-Beta pruning, Monte Carlo Tree Search, or heuristic cutoffs)
- Non-zero-sum games (both can benefit)
- Games with chance (dice, cards) — use Expectiminimax
- Games with hidden information (poker) — use imperfect information algorithms
- Single-player games — use DP or BFS/DFS
- Problems that don't require recursive opponent modeling

5. Core Concepts

Game Tree

A tree where each node is a game state, edges are moves. Levels alternate between Max and Min.

Terminal / Leaf Node

A state where the game ends. Assigned a score directly (win/lose/draw or numeric evaluation).

Max Node

A node where the current player aims to maximize the score. The value is the maximum of its children's minimax values.

Min Node

A node where the opponent aims to minimize the score. The value is the minimum of its children's minimax values.

Evaluation Function

A heuristic that estimates the score of a non-terminal state. Used when we cannot expand the full tree (depth-limited minimax).

Minimax Value

The score of a state under optimal play by both sides. Propagated from leaves up to root.

Zero-Sum Game

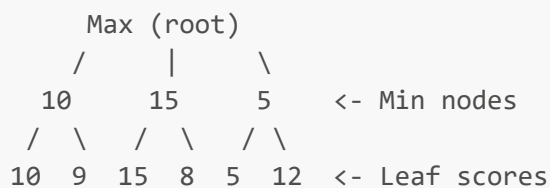
The total gain of both players sums to zero. Max's gain = Min's loss. Minimax assumes this.

6. Step-by-Step Algorithm

1. **Define the state representation** for the game.
2. **Define terminal condition** and evaluation function.
3. **Define legal moves** from any state.
4. **Implement minimax:**
 - If state is terminal → return evaluation.
 - If it's Max's turn:
 - Initialize `best = -INF`.
 - For each legal move:
 - `best = max(best, minimax(next_state, depth-1, false))`.
 - Return `best`.
 - If it's Min's turn:
 - Initialize `best = +INF`.
 - For each legal move:
 - `best = min(best, minimax(next_state, depth-1, true))`.
 - Return `best`.
5. **At the root:** Choose the move that gives the highest minimax value (if Max's turn) or lowest (if Min's turn).

7. Dry Run

Problem: Simple game tree with depth 2.



Step 1: Leaf values are given.

Step 2: Min layer (depth 1):

- Left child: $\min(10, 9) = 9$
- Middle child: $\min(15, 8) = 8$
- Right child: $\min(5, 12) = 5$

Step 3: Root (Max, depth 0):

- $\max(9, 8, 5) = 9$

Result: Minimax value = 9. Optimal first move: go to left child (score 9 guaranteed).

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// Tic-Tac-Toe minimax example
const int INF = 1e9;
const int BOARD_SIZE = 3;

// Evaluate the board: +10 for Max win, -10 for Min win, 0 for draw, 0 for ongoing
int evaluate(const vector<vector<char>>& board, char maxChar, char minChar) {
    // Check rows
    for (int i = 0; i < BOARD_SIZE; i++) {
        if (board[i][0] == board[i][1] && board[i][1] == board[i][2]) {
            if (board[i][0] == maxChar) return 10;
            if (board[i][0] == minChar) return -10;
        }
    }
    // Check columns
    for (int j = 0; j < BOARD_SIZE; j++) {
        if (board[0][j] == board[1][j] && board[1][j] == board[2][j]) {
            if (board[0][j] == maxChar) return 10;
            if (board[0][j] == minChar) return -10;
        }
    }
    // Check diagonals
    if (board[0][0] == board[1][1] && board[1][1] == board[2][2]) {
        if (board[0][0] == maxChar) return 10;
        if (board[0][0] == minChar) return -10;
    }
    if (board[0][2] == board[1][1] && board[1][1] == board[2][0]) {
        if (board[0][2] == maxChar) return 10;
        if (board[0][2] == minChar) return -10;
    }
    return 0; // no winner
}

// Check if there are moves left
bool hasMovesLeft(const vector<vector<char>>& board) {
    for (int i = 0; i < BOARD_SIZE; i++)
        for (int j = 0; j < BOARD_SIZE; j++)
            if (board[i][j] == ' ') return true;
    return false;
}

// Minimax algorithm
int minimax(vector<vector<char>>& board, int depth, bool isMax,
            char maxChar, char minChar) {
    int score = evaluate(board, maxChar, minChar);

    // Terminal: Max wins
    if (score == 10) return score - depth; // prefer faster wins
    // Terminal: Min wins
    if (score == -10) return score + depth; // prefer faster wins (more negative

```

```

sooner)
// Terminal: draw
if (!hasMovesLeft(board)) return 0;

if (isMax) {
    int best = -INF;
    for (int i = 0; i < BOARD_SIZE; i++) {
        for (int j = 0; j < BOARD_SIZE; j++) {
            if (board[i][j] == ' ') {
                board[i][j] = maxChar;
                best = max(best, minimax(board, depth + 1, false, maxChar,
minChar));
                board[i][j] = ' '; // undo
            }
        }
    }
    return best;
} else {
    int best = INF;
    for (int i = 0; i < BOARD_SIZE; i++) {
        for (int j = 0; j < BOARD_SIZE; j++) {
            if (board[i][j] == ' ') {
                board[i][j] = minChar;
                best = min(best, minimax(board, depth + 1, true, maxChar,
minChar));
                board[i][j] = ' ';
            }
        }
    }
    return best;
}
}

// Find best move for Max player
pair<int, int> findBestMove(vector<vector<char>>& board, char maxChar, char
minChar) {
    int bestVal = -INF;
    pair<int, int> bestMove = {-1, -1};

    for (int i = 0; i < BOARD_SIZE; i++) {
        for (int j = 0; j < BOARD_SIZE; j++) {
            if (board[i][j] == ' ') {
                board[i][j] = maxChar;
                int moveVal = minimax(board, 0, false, maxChar, minChar);
                board[i][j] = ' ';
                if (moveVal > bestVal) {
                    bestVal = moveVal;
                    bestMove = {i, j};
                }
            }
        }
    }

    return bestMove;
}

```

```

}

int main() {
    // Empty board
    vector<vector<char>> board(BOARD_SIZE, vector<char>(BOARD_SIZE, ' '));

    auto [r, c] = findBestMove(board, 'X', 'O');
    cout << "Best move for X: (" << r << ", " << c << ")\n";

    return 0;
}

```

9. Python Implementation

```

from typing import List, Tuple

INF = 10 ** 9
BOARD_SIZE = 3

def evaluate(board: List[List[str]], max_char: str, min_char: str) -> int:
    """Evaluate board: +10 for Max win, -10 for Min win, 0 otherwise."""
    # Rows
    for i in range(BOARD_SIZE):
        if board[i][0] == board[i][1] == board[i][2]:
            if board[i][0] == max_char:
                return 10
            if board[i][0] == min_char:
                return -10

    # Columns
    for j in range(BOARD_SIZE):
        if board[0][j] == board[1][j] == board[2][j]:
            if board[0][j] == max_char:
                return 10
            if board[0][j] == min_char:
                return -10

    # Diagonals
    if board[0][0] == board[1][1] == board[2][2]:
        if board[0][0] == max_char:
            return 10
        if board[0][0] == min_char:
            return -10

    if board[0][2] == board[1][1] == board[2][0]:
        if board[0][2] == max_char:
            return 10
        if board[0][2] == min_char:
            return -10

```

```

    return 0

def has_moves_left(board: List[List[str]]) -> bool:
    for i in range(BOARD_SIZE):
        for j in range(BOARD_SIZE):
            if board[i][j] == ' ':
                return True
    return False

def minimax(board: List[List[str]], depth: int, is_max: bool,
            max_char: str, min_char: str) -> int:
    score = evaluate(board, max_char, min_char)

    if score == 10: # Max wins
        return score - depth
    if score == -10: # Min wins
        return score + depth
    if not has_moves_left(board): # Draw
        return 0

    if is_max:
        best = -INF
        for i in range(BOARD_SIZE):
            for j in range(BOARD_SIZE):
                if board[i][j] == ' ':
                    board[i][j] = max_char
                    best = max(best, minimax(board, depth + 1, False, max_char,
min_char))
                    board[i][j] = ' '
            return best
    else:
        best = INF
        for i in range(BOARD_SIZE):
            for j in range(BOARD_SIZE):
                if board[i][j] == ' ':
                    board[i][j] = min_char
                    best = min(best, minimax(board, depth + 1, True, max_char,
min_char))
                    board[i][j] = ' '
            return best

def find_best_move(board: List[List[str]], max_char: str, min_char: str) ->
Tuple[int, int]:
    best_val = -INF
    best_move = (-1, -1)

    for i in range(BOARD_SIZE):
        for j in range(BOARD_SIZE):
            if board[i][j] == ' ':
                board[i][j] = max_char
                move_val = minimax(board, 0, False, max_char, min_char)

```

```

        board[i][j] = ' '
        if move_val > best_val:
            best_val = move_val
            best_move = (i, j)

    return best_move

if __name__ == "__main__":
    board = [[' ' for _ in range(BOARD_SIZE)] for _ in range(BOARD_SIZE)]
    r, c = find_best_move(board, 'X', 'O')
    print(f"Best move for X: ({r}, {c})")

```

10. Code Explanation

Key parts of the code:

1. **evaluate**: Scores a board state. Returns +10 for Max win, -10 for Min win, 0 for no winner (or draw). Checks rows, columns, and diagonals.
2. **minimax**: Recursive function:
 - Base case: terminal state → return adjusted score (depth adjustment prefers faster wins).
 - Max turn: for each move, compute minimax of resulting state, take max.
 - Min turn: for each move, compute minimax of resulting state, take min.
 - Uses backtracking: make move, recurse, undo move.
3. **findBestMove**: Tries each possible move from current state (Max's turn), calls minimax for each, selects the move with highest value.

Depth adjustment: **score - depth** for Max wins makes closer wins score higher. **score + depth** for Min wins makes closer wins for Min score more negative.

Edge cases handled: Full board (draw), immediate win detected, immediate loss avoided.

11. Complexity Analysis

Factor	Complexity
Game tree size	$O(b^d)$ where b = branching factor, d = depth
Time	$O(b^d)$
Space	$O(d)$ (recursion depth)

b = branching factor (average number of legal moves per state). **d = depth** of game tree (or search depth limit).

For Tic-Tac-Toe: $b \approx 4-9$, $d \leq 9 \rightarrow$ manageable. For Chess: $b \approx 35$, $d \approx 80 \rightarrow 35^{80}$ is impossible $\rightarrow$ need alpha-beta pruning + depth limits.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Standard Minimax	Two-player zero-sum perfect info	Recursive min/max over children	Tic-Tac-Toe, Connect Four
Minimax with DP	Overlapping subgames	Memoize (state → minimax value)	Small-board games
Depth-limited Minimax	Large state space	Cut off at depth D, use heuristic	Chess engines
Minimax with Move Ordering	Need faster search	Sort moves by heuristic quality	Used with alpha-beta
Iterative Deepening	Unknown optimal depth	Search depth 1, 2, 3,... until time limit	Tournament game AI

13. Common Mistakes

- Not negating the evaluation for the opponent (evaluation should be from Max's perspective).
- Incorrect terminal condition — missing draw states.
- Depth adjustment wrong — adding instead of subtracting for faster wins.
- Integer overflow in evaluation — use int, not unsigned.
- Forgetting to undo moves in backtracking — corrupts state.
- Using evaluation function that's not symmetric (positive for Max, negative for Min).
- Infinite recursion if game can loop — need visited set or depth limit.
- Not handling the case where no moves exist from a non-terminal state.

14. Edge Cases

- Empty board → first move.
- Board with one move left → should pick win/draw.
- Board where all moves lead to loss → any move equally bad.
- Draw state → evaluation 0.
- Immediate win available → should take it (depth adjustment helps).
- Immediate loss unavoidable → pick least damaging move.

15. Variations

Negamax

A simplification of minimax. Instead of separate max/min functions, negate the evaluation at each level. Both players run the same code: `value = -minimax(child)`.

```
function negamax(state, depth):
    if terminal(state): return evaluate(state)
    best = -INF
    for move in legal_moves(state):
```

```
best = max(best, -negamax(next_state, depth-1))
return best
```

Expectiminimax

For games with chance (dice, shuffled cards). Adds "chance nodes" where the value is the expected value (average) of outcomes.

Depth-Limited Minimax

Used when full tree is too large. Search to depth D, then use heuristic evaluation. Common in chess engines.

Minimax with Alpha-Beta Pruning

(See next section.) Dramatically reduces the number of nodes evaluated.

16. Related Algorithms/Data Structures

Topic	Connection
Alpha-Beta Pruning	Optimization of minimax, skips irrelevant branches.
Negamax	Simplified implementation of minimax.
Game Theory (Win/Lose)	Minimax for win/lose games (scores are just WIN/LOSE).
Expectiminimax	Minimax with chance nodes.
Monte Carlo Tree Search	For very large games where minimax is infeasible.
Dynamic Programming	Used to memoize minimax values for identical states.

17. Practice Problems

Easy

1. **Nim Game** — LeetCode 292. Simple minimax with win/lose scores.
2. **Predict the Winner** — LeetCode 486. Minimax on array picking from ends.

Medium

1. **Stone Game** — LeetCode 877. Minimax on even-length piles.
2. **Can I Win** — LeetCode 464. Minimax with memoization on bitmask.
3. **Tic-Tac-Toe** — LeetCode 348 / GFG. Design a Tic-Tac-Toe with minimax AI.

Hard

1. **Cat and Mouse** — LeetCode 913. Minimax on graph with states.
2. **Chess Engine** — Not a single problem, but implementing a basic chess AI uses depth-limited minimax + alpha-beta.

18. Interview Explanation

"Minimax is a recursive algorithm for two-player zero-sum games with perfect information. It assumes both players play optimally. The Max player maximizes the score, and the Min player minimizes it. At each state, we compute the minimax value: Max picks the maximum of children, Min picks the minimum. Terminal states have known scores. The algorithm explores the entire game tree, which gives $O(b^d)$ complexity. For large games, we use alpha-beta pruning to cut branches and depth-limited search with heuristic evaluation. Minimax is used in games like Tic-Tac-Toe, Connect Four, and Chess endgames."

19. Revision Notes

- **Max turn:** `best = max(child values)`
- **Min turn:** `best = min(child values)`
- **Terminal:** return evaluation score
- **$O(b^d)$ time** — use pruning for large trees
- **Negamax:** `best = max(-negamax(child))`
- **Depth adjustment:** prefer faster wins, slower losses
- **Backtracking:** always undo moves after recursion

20. Final Cheat Sheet

Item	Detail
When to use	Two-player zero-sum, perfect info, small-enough game tree
Main operation	Max picks max, Min picks min over children
Formula	Max: <code>max(minimax(child))</code> ; Min: <code>min(minimax(child))</code>
Code pattern	<code>if isMax: best = max(best, minimax(...)) else: best = min(best, minimax(...))</code>
Complexity	$O(b^d)$ time, $O(d)$ space
Key trick	Depth adjustment for faster wins
Edge cases	Empty board, draw, immediate win, no moves
Trap	Forgetting to undo moves; asymmetric evaluation

ALPHA-BETA PRUNING

1. Overview

Alpha-beta pruning is an optimization of the minimax algorithm. It prunes (cuts off) branches of the game tree that cannot possibly influence the final decision, dramatically reducing the number of nodes evaluated while still computing the exact same result as full minimax.

2. Intuition

Simple explanation: Alpha-beta pruning says: "If I already know a move gives me at least X , and a branch I'm exploring can only give me at most $Y < X$, I can stop exploring that branch — it's not going to beat what I already have."

Analogy: You're shopping for a laptop with a friend. You find one for \$800. Your friend says "I found one for \$900 at the next store, but let me check the other brands there." You say "Stop — \$900 is worse than \$800, I don't need to hear about other brands at that store." That's alpha-beta pruning.

Step-by-step reasoning:

1. At a Max node, maintain **alpha**: the best (maximum) value found so far along the current path.
2. At a Min node, maintain **beta**: the best (minimum) value found so far along the current path.
3. Initially, $\alpha = -\infty$, $\beta = +\infty$.
4. While exploring children:
 - At a Max node: update $\alpha = \max(\alpha, \text{child_value})$. If $\alpha \geq \beta$, prune rest.
 - At a Min node: update $\beta = \min(\beta, \text{child_value})$. If $\beta \leq \alpha$, prune rest.
5. Pruned branches are not explored; the result is identical to minimax.

Why it works: The alpha value is the minimum that Max is guaranteed. The beta value is the maximum that Min is guaranteed. If $\alpha \geq \beta$, there's no overlap — the current branch can't affect the decision, so it's safe to ignore.

3. When to Use It

- Any situation where you would use minimax but the game tree is too large
- Games with high branching factor (Chess: $b \approx 35$)
- Real-time game AI where computation time is limited
- Problems where minimax runs but you need to optimize
- Combined with depth-limited search and iterative deepening

Common trigger phrases: "optimize minimax", "game tree too large", "pruning", "alpha-beta", "improve search efficiency", "game AI optimization".

4. When Not to Use It

- The game tree is already small enough for full minimax (no benefit from pruning)
- Not a two-player zero-sum perfect-information game
- When move ordering is poor (pruning is most effective with good move ordering)
- Expectiminimax (games with chance) — alpha-beta doesn't directly apply
- Non-game problems (use other optimization techniques)

5. Core Concepts

Alpha (α)

The best (highest) value that Max can guarantee along the current path. Initialized to $-\infty$. Updated at Max nodes.

Beta (β)

The best (lowest) value that Min can guarantee along the current path. Initialized to $+\infty$. Updated at Min nodes.

Pruning Condition

- At a Max node: if $\alpha \geq \beta$, prune remaining children.
- At a Min node: if $\beta \leq \alpha$, prune remaining children.

Search Window

The interval $[\alpha, \beta]$. As the search progresses, this window narrows. When it becomes empty ($\alpha \geq \beta$), pruning occurs.

Move Ordering

The order in which moves are explored significantly affects pruning efficiency. Best moves first gives more pruning. This is why chess engines spend time on move ordering before search.

Effectiveness

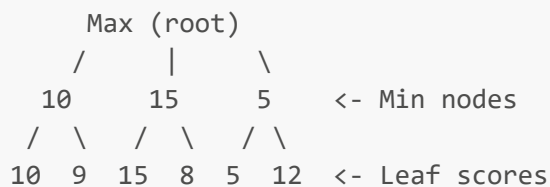
With perfect move ordering, alpha-beta reduces complexity from $O(b^d)$ to $O(b^{(d/2)})$ — effectively doubling the searchable depth.

6. Step-by-Step Algorithm

1. **Initialize** $\alpha = -\text{INF}$, $\beta = +\text{INF}$ at root.
2. **Call alpha-beta** on root state.
3. **At each node:**
 - If terminal $\rightarrow$ return evaluation.
 - If Max node:
 - $\text{value} = -\text{INF}$
 - For each child:
 - $\text{value} = \max(\text{value}, \text{alpha_beta}(\text{child}, \alpha, \beta, \text{false}))$
 - $\alpha = \max(\alpha, \text{value})$
 - If $\alpha \geq \beta \rightarrow$ **prune**: break (return value early).
 - Return value .
 - If Min node:
 - $\text{value} = +\text{INF}$
 - For each child:
 - $\text{value} = \min(\text{value}, \text{alpha_beta}(\text{child}, \alpha, \beta, \text{true}))$
 - $\beta = \min(\beta, \text{value})$
 - If $\beta \leq \alpha \rightarrow$ **prune**: break.
 - Return value .

7. Dry Run

Problem: Same tree as minimax example.



Step 1: Call alpha-beta on root with $\alpha = -\infty$, $\beta = +\infty$.

Step 2: Explore left child (Min node). Pass $\alpha = -\infty$, $\beta = +\infty$.

- Explore leaf 10: value = 10. $\beta = \min(+\infty, 10) = 10$.
- Explore leaf 9: value = $\min(10, 9) = 9$. $\beta = \min(10, 9) = 9$.
- $\beta = 9 > \alpha = -\infty \rightarrow$ no prune.
- Return 9. $\alpha = \max(-\infty, 9) = 9$.

Step 3: Explore middle child (Min node). Pass $\alpha = 9$, $\beta = +\infty$.

- Explore leaf 15: value = 15. $\beta = \min(+\infty, 15) = 15$.
- $\alpha = 9$, $\beta = 15 \rightarrow$ no prune.
- Explore leaf 8: value = $\min(15, 8) = 8$. $\beta = \min(15, 8) = 8$.
- Now $\beta = 8$, $\alpha = 9 \rightarrow \beta \leq \alpha \rightarrow$ **PRUNE**. No need to check right branch of middle child (it's already worse for Max than what we have from left child).
- Return 8.

$\alpha = \max(9, 8) = 9$ (unchanged).

Step 4: Explore right child (Min node). Pass $\alpha = 9$, $\beta = +\infty$.

- Explore leaf 5: value = 5. $\beta = \min(+\infty, 5) = 5$.
- $\beta = 5$, $\alpha = 9 \rightarrow \beta \leq \alpha \rightarrow$ **PRUNE**. Entire right subtree is pruned.
- Return 5.

$\alpha = \max(9, 5) = 9$.

Result: Same as minimax (9), but skipped exploring leaf 12 and possibly other subtrees.

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;
const int BOARD_SIZE = 3;

// Evaluation function (same as minimax)
int evaluate(const vector<vector<char>>& board, char maxChar, char minChar) {
    for (int i = 0; i < BOARD_SIZE; i++) {
        if (board[i][0] == board[i][1] && board[i][1] == board[i][2]) {
            if (board[i][0] == maxChar) return 10;
        }
    }
    // ... (rest of the implementation)
}
  
```

```

        if (board[i][0] == minChar) return -10;
    }
}
for (int j = 0; j < BOARD_SIZE; j++) {
    if (board[0][j] == board[1][j] && board[1][j] == board[2][j]) {
        if (board[0][j] == maxChar) return 10;
        if (board[0][j] == minChar) return -10;
    }
}
if (board[0][0] == board[1][1] && board[1][1] == board[2][2]) {
    if (board[0][0] == maxChar) return 10;
    if (board[0][0] == minChar) return -10;
}
if (board[0][2] == board[1][1] && board[1][1] == board[2][0]) {
    if (board[0][2] == maxChar) return 10;
    if (board[0][2] == minChar) return -10;
}
return 0;
}

bool hasMovesLeft(const vector<vector<char>>& board) {
    for (int i = 0; i < BOARD_SIZE; i++)
        for (int j = 0; j < BOARD_SIZE; j++)
            if (board[i][j] == ' ') return true;
    return false;
}

// Alpha-beta pruning
int alphaBeta(vector<vector<char>>& board, int depth, int alpha, int beta,
              bool isMax, char maxChar, char minChar) {
    int score = evaluate(board, maxChar, minChar);

    if (score == 10) return score - depth;
    if (score == -10) return score + depth;
    if (!hasMovesLeft(board)) return 0;

    if (isMax) {
        int value = -INF;
        for (int i = 0; i < BOARD_SIZE; i++) {
            for (int j = 0; j < BOARD_SIZE; j++) {
                if (board[i][j] == ' ') {
                    board[i][j] = maxChar;
                    value = max(value, alphaBeta(board, depth + 1, alpha, beta,
false, maxChar, minChar));
                    board[i][j] = ' ';
                    alpha = max(alpha, value);
                    if (alpha >= beta) return value; // prune
                }
            }
        }
        return value;
    } else {
        int value = INF;
        for (int i = 0; i < BOARD_SIZE; i++) {

```

```

        for (int j = 0; j < BOARD_SIZE; j++) {
            if (board[i][j] == ' ') {
                board[i][j] = minChar;
                value = min(value, alphaBeta(board, depth + 1, alpha, beta,
true, maxChar, minChar));
                board[i][j] = ' ';
                beta = min(beta, value);
                if (beta <= alpha) return value; // prune
            }
        }
    }
    return value;
}

// Find best move using alpha-beta pruning
pair<int, int> findBestMove(vector<vector<char>>& board, char maxChar, char
minChar) {
    int bestVal = -INF;
    pair<int, int> bestMove = {-1, -1};

    for (int i = 0; i < BOARD_SIZE; i++) {
        for (int j = 0; j < BOARD_SIZE; j++) {
            if (board[i][j] == ' ') {
                board[i][j] = maxChar;
                int moveVal = alphaBeta(board, 0, -INF, INF, false, maxChar,
minChar);
                board[i][j] = ' ';
                if (moveVal > bestVal) {
                    bestVal = moveVal;
                    bestMove = {i, j};
                }
            }
        }
    }

    return bestMove;
}

int main() {
    vector<vector<char>> board(BOARD_SIZE, vector<char>(BOARD_SIZE, ' '));

    auto [r, c] = findBestMove(board, 'X', 'O');
    cout << "Best move for X (alpha-beta): (" << r << ", " << c << ")\n";

    return 0;
}

```

9. Python Implementation

```

from typing import List, Tuple

INF = 10 ** 9
BOARD_SIZE = 3

def evaluate(board: List[List[str]], max_char: str, min_char: str) -> int:
    for i in range(BOARD_SIZE):
        if board[i][0] == board[i][1] == board[i][2]:
            if board[i][0] == max_char:
                return 10
            if board[i][0] == min_char:
                return -10
    for j in range(BOARD_SIZE):
        if board[0][j] == board[1][j] == board[2][j]:
            if board[0][j] == max_char:
                return 10
            if board[0][j] == min_char:
                return -10
    if board[0][0] == board[1][1] == board[2][2]:
        if board[0][0] == max_char:
            return 10
        if board[0][0] == min_char:
            return -10
    if board[0][2] == board[1][1] == board[2][0]:
        if board[0][2] == max_char:
            return 10
        if board[0][2] == min_char:
            return -10
    return 0

def has_moves_left(board: List[List[str]]) -> bool:
    for i in range(BOARD_SIZE):
        for j in range(BOARD_SIZE):
            if board[i][j] == ' ':
                return True
    return False

def alpha_beta(board: List[List[str]], depth: int, alpha: int, beta: int,
               is_max: bool, max_char: str, min_char: str) -> int:
    score = evaluate(board, max_char, min_char)

    if score == 10:
        return score - depth
    if score == -10:
        return score + depth
    if not has_moves_left(board):
        return 0

    if is_max:

```

```

        value = -INF
        for i in range(BOARD_SIZE):
            for j in range(BOARD_SIZE):
                if board[i][j] == ' ':
                    board[i][j] = max_char
                    value = max(value, alpha_beta(board, depth + 1, alpha, beta,
False, max_char, min_char))
                    board[i][j] = ' '
                    alpha = max(alpha, value)
                    if alpha >= beta:
                        return value # prune

            return value
    else:
        value = INF
        for i in range(BOARD_SIZE):
            for j in range(BOARD_SIZE):
                if board[i][j] == ' ':
                    board[i][j] = min_char
                    value = min(value, alpha_beta(board, depth + 1, alpha, beta,
True, max_char, min_char))
                    board[i][j] = ' '
                    beta = min(beta, value)
                    if beta <= alpha:
                        return value # prune

            return value

def find_best_move(board: List[List[str]], max_char: str, min_char: str) ->
Tuple[int, int]:
    best_val = -INF
    best_move = (-1, -1)

    for i in range(BOARD_SIZE):
        for j in range(BOARD_SIZE):
            if board[i][j] == ' ':
                board[i][j] = max_char
                move_val = alpha_beta(board, 0, -INF, INF, False, max_char,
min_char)
                board[i][j] = ' '
                if move_val > best_val:
                    best_val = move_val
                    best_move = (i, j)

    return best_move

if __name__ == "__main__":
    board = [[' ' for _ in range(BOARD_SIZE)] for _ in range(BOARD_SIZE)]
    r, c = find_best_move(board, 'X', 'O')
    print(f"Best move for X (alpha-beta): ({r}, {c})")

```

10. Code Explanation

Key differences from minimax:

1. **Alpha and beta parameters:** `alpha` (best for Max so far) and `beta` (best for Min so far) are passed down the recursion.
2. **Pruning at Max node:**
 - Update `alpha = max(alpha, value)`.
 - If `alpha >= beta`, prune: `return value` immediately.
3. **Pruning at Min node:**
 - Update `beta = min(beta, value)`.
 - If `beta <= alpha`, prune: `return value` immediately.
4. **Same evaluation and terminal checks** as minimax.

Why pruning is safe: The alpha value represents a lower bound (Max's guarantee). The beta value represents an upper bound (Min's guarantee). If the bounds cross ($\alpha \geq \beta$), the current node cannot affect the root's decision — exploring further is pointless.

11. Complexity Analysis

Case	Time	Space
Minimax (no pruning)	$O(b^d)$	$O(d)$
Alpha-beta (best case)	$O(b^{(d/2)})$	$O(d)$
Alpha-beta (average)	$O(b^{(3d/4)})$	$O(d)$
Alpha-beta (worst)	$O(b^d)$	$O(d)$

Best case: Perfect move ordering (best move explored first at each node). This doubles the searchable depth.

Worst case: Worst move ordering first (worst case explored first). Degrades to full minimax.

Effective branching factor: With good move ordering, $\sqrt{b}$ instead of b .

Move ordering techniques: Captures first (in Chess), killer moves, history heuristic, transposition table.

12. Common Patterns

Pattern	How to Identify	Approach	Example Problems
Standard Alpha-Beta	Two-player zero-sum, want efficiency	Basic alpha-beta with move ordering	Tic-Tac-Toe, Connect Four
Alpha-Beta with Iterative Deepening	Need best move under time limit	Search depth 1, 2, 3... until timeout	Chess engines
Alpha-Beta with Transposition Table	Same state reached through different paths	Cache (state → value, alpha, beta) in hash table	Chess, Go endgames

Pattern	How to Identify	Approach	Example Problems
Zero-Window Search	Want to test if value > threshold quickly	Set $\alpha = \beta - 1$, returns bounds	Scout / Principal Variation Search
Aspiration Search	Narrow search window initially	Start with [best-known - margin, best-known + margin]	Tournament game AI

13. Common Mistakes

- Forgetting to update alpha/beta correctly at each level.
- Pruning at the wrong node type (pruning at Max should check $\alpha \geq \beta$, not $\beta \leq \alpha$).
- Not passing updated alpha/beta to children correctly.
- Pruning too aggressively — trust the algorithm, it's provably correct.
- Poor move ordering leading to minimal pruning — invest in good ordering.
- Off-by-one in depth limits causing incorrect evaluation.
- Using alpha-beta on non-zero-sum games (results are meaningless).

14. Edge Cases

- Very first child explored: alpha and beta are still $-\infty/\infty$, no pruning happens yet.
- All children pruned: return current value (which is the value of the first child explored).
- Immediate win: return early with win score, prune everything else.
- Draw: all values near 0, pruning may be minimal.
- Identical sibling values: pruning still works (alpha/beta update preserves best).

15. Variations

Negamax with Alpha-Beta

Combines negamax simplification with alpha-beta pruning.

```
function negamaxAB(state, depth, alpha, beta):
    if terminal(state): return evaluate(state)
    for each move in order_moves(state):
        value = -negamaxAB(next_state, depth-1, -beta, -alpha)
        alpha = max(alpha, value)
        if alpha >= beta: break
    return alpha
```

Principal Variation Search (PVS)

A refinement of alpha-beta that uses zero-window searches on non-PV nodes to reduce the search tree further. Used in modern chess engines.

Aspiration Search

Instead of starting with $[-\text{INF}, +\text{INF}]$, start with a narrow window $[\text{best} - \text{margin}, \text{best} + \text{margin}]$. If the search fails (value outside window), re-search with wider window. Saves time when the value is close to expected.

MTD(f)

An even more aggressive search that uses repeated zero-window searches to narrow down the exact value. Used in some game engines.

16. Related Algorithms/Data Structures

Topic	Connection
Minimax	The base algorithm that alpha-beta optimizes.
Negamax	Simplified minimax that pairs well with alpha-beta.
Iterative Deepening	Combined with alpha-beta for time-bounded search.
Transposition Table	Hash table storing evaluated states to avoid re-computation.
Move Ordering	Critical for alpha-beta efficiency.
Game Theory	The domain where all these algorithms live.

17. Practice Problems

Easy

1. **Nim Game** — LeetCode 292. Implement minimax, then add alpha-beta.
2. **Predict the Winner** — LeetCode 486. Add alpha-beta to optimise.

Medium

1. **Stone Game** — LeetCode 877. Minimax with alpha-beta.
2. **Can I Win** — LeetCode 464. Minimax + memoization + alpha-beta.
3. **Tic-Tac-Toe AI** — GFG. Build a full Tic-Tac-Toe AI with alpha-beta.

Hard

1. **Connect Four AI** — Build a Connect Four AI with alpha-beta + iterative deepening.
2. **Othello (Reversi) AI** — Implement alpha-beta with heuristics.
3. **Chess Endgame** — Codeforces / Custom. Use alpha-beta with depth-limited search.

18. Interview Explanation

"Alpha-beta pruning is an optimization of minimax that reduces the number of nodes explored without changing the result. It maintains two values: alpha, the best value Max can guarantee so far, and beta, the best value Min can guarantee so far. At a Max node, if alpha becomes $\geq$ beta, we prune remaining children because Min would never allow the game to reach that branch. At a Min node, if beta $\leq$ alpha, we prune. With perfect move ordering, alpha-beta reduces complexity from $O(b^d)$ to $O(b^{(d/2)})$, effectively doubling the searchable depth. In practice, good move ordering — like captures first or using a transposition table — is essential for alpha-beta to be effective."

19. Revision Notes

- α : best for Max (lower bound), initialized $-\infty$
- β : best for Min (upper bound), initialized $+\infty$
- **Prune at Max**: if $\alpha \geq \beta$, break
- **Prune at Min**: if $\beta \leq \alpha$, break
- **Move ordering** is critical for efficiency
- **Best case** $O(b^{(d/2)})$ vs minimax $O(b^d)$
- **Same result** as minimax — exact, not approximate
- **Negamax version**: `alpha = max(alpha, -negamax(child, -beta, -alpha))`

20. Final Cheat Sheet

Item	Detail
When to use	Optimizing minimax for large game trees
Main operation	Prune when $\alpha \geq \beta$ (Max) or $\beta \leq \alpha$ (Min)
Parameters	<code>alpha</code> = best for Max, <code>beta</code> = best for Min
Code pattern	<code>if (isMax) { alpha = max(alpha, val); if (alpha >= beta) break; }</code>
Complexity	Best: $O(b^{(d/2)})$; Worst: $O(b^d)$
Key enabler	Good move ordering (best moves first)
Variations	PVS, Aspiration Search, MTD(f), Negamax-AB
Trap	Forgetting to update α/β before pruning check; prunes too early with bad ordering

End of Guide — Game Theory Algorithms for Placements & CP